

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Jacob Devlin Ming-Wei Chang Kenton Lee Kristina Toutanova
Google AI Language
{jacobdevlin, mingweichang, kentonl, kristout}@google.com

Abstract

We introduce a new language representation model called **BERT**, which stands for **B**idirectional **E**ncoder **R**epresentations from **T**ransformers. Unlike recent language representation models (Peters et al., 2018a; Radford et al., 2018), BERT is designed to pre-train deep bidirectional representations from unlabeled text by jointly conditioning on both left and right context in all layers. As a result, the pre-trained BERT model can be fine-tuned with just one additional output layer to create state-of-the-art models for a wide range of tasks, such as question answering and language inference, without substantial task-specific architecture modifications.

BERT is conceptually simple and empirically powerful. It obtains new state-of-the-art results on eleven natural language processing tasks, including pushing the GLUE score to 80.5% (7.7% point absolute improvement), MultiNLI accuracy to 86.7% (4.6% absolute improvement), SQuAD v1.1 question answering Test F1 to 93.2 (1.5 point absolute improvement) and SQuAD v2.0 Test F1 to 83.1 (5.1 point absolute improvement).

1 Introduction

Language model pre-training has been shown to be effective for improving many natural language processing tasks (Dai and Le, 2015; Peters et al., 2018a; Radford et al., 2018; Howard and Ruder, 2018). These include sentence-level tasks such as natural language inference (Bowman et al., 2015; Williams et al., 2018) and paraphrasing (Dolan and Brockett, 2005), which aim to predict the relationships between sentences by analyzing them holistically, as well as token-level tasks such as named entity recognition and question answering, where models are required to produce fine-grained output at the token level (Tjong Kim Sang and De Meulder, 2003; Rajpurkar et al., 2016).

There are two existing strategies for applying pre-trained language representations to downstream tasks: *feature-based* and *fine-tuning*. The feature-based approach, such as ELMo (Peters et al., 2018a), uses task-specific architectures that include the pre-trained representations as additional features. The fine-tuning approach, such as the Generative Pre-trained Transformer (OpenAI GPT) (Radford et al., 2018), introduces minimal task-specific parameters, and is trained on the downstream tasks by simply fine-tuning *all* pre-trained parameters. The two approaches share the same objective function during pre-training, where they use unidirectional language models to learn general language representations.

We argue that current techniques restrict the power of the pre-trained representations, especially for the fine-tuning approaches. The major limitation is that standard language models are unidirectional, and this limits the choice of architectures that can be used during pre-training. For example, in OpenAI GPT, the authors use a left-to-right architecture, where every token can only attend to previous tokens in the self-attention layers of the Transformer (Vaswani et al., 2017). Such restrictions are sub-optimal for sentence-level tasks, and could be very harmful when applying fine-tuning based approaches to token-level tasks such as question answering, where it is crucial to incorporate context from both directions.

In this paper, we improve the fine-tuning based approaches by proposing BERT: **B**idirectional **E**ncoder **R**epresentations from **T**ransformers. BERT alleviates the previously mentioned unidirectionality constraint by using a “masked language model” (MLM) pre-training objective, inspired by the Cloze task (Taylor, 1953). The masked language model randomly masks some of the tokens from the input, and the objective is to predict the original vocabulary id of the masked

BERT：用于语言理解的深度双向Transformer预训练

Jacob Devlin、Ming-Wei Chang、Kenton Lee与Kristina Toutanova谷歌
AI语言团队{jacobdevlin, mingweichang, kentonl, kristout}@google.com

摘要

我们提出了一种新的语言表示模型，名为**BERT**，全称为双向**编**码器特征由Transformer提供。与近期Peters等, 2018a;Radford等, 2018的研究不同，BERT旨在通过在所有层中联合考虑左右上下文，从未标注文本中预训练深度双向表征。因此，预训练的BERT模型只需添加一个输出层即可微调，从而为各类任务（如问答和语言推理）构建最先进的模型，且无需对特定任务架构进行重大改动。

BERT在概念上十分简洁，且在实证效果上极为强大。它在十一项自然语言处理任务上取得了全新的最先进水平结果，包括将GLUE评分提升至80.5%（绝对提升7.7个百分点）、MultiNLI准确率达到86.7%（绝对提升4.6个百分点）、SQuAD v1.1问答测试F1得分达93.2（绝对提升1.5分）以及SQuAD v2.0测试F1得分达83.1（绝对提升5.1分）。

1 引言

语言模型预训练已被证实能有效提升多项自然语言处理任务的性能（Dai and Le, 2015;

Peters et al., 2018a; Radford et al., 2018; Howard and Ruder, 2018）。其中包括句子级任务，如自然语言推理（Bowman et

al., 2015; Williams et al., 2018）以及文本改写（Dolan and Brockett, 2005），旨在通过整体分析预测句子间的关系；同时还包括命名实体识别与问答等词元级任务，此类任务要求模型在词元层面生成细粒度的输出（Tjong Kim Sang and De Meulder, 2003; Rajpurkar et al., 2016）。

目前有两种现有策略可将预训练语言表示应用于下游任务：基于特征的方法以及微调。基于特征的方法（如ELMo（Peters等人, 2018a）），采用特定于任务的架构，并将预训练表示作为附加特征包含在内。微调方法（如生成式预训练转换器（OpenAI GPT）（Radford等人, 2018）），仅引入极少的特定任务参数，并通过对所有预训练参数进行微调来在下游任务上进行训练。这两种方法在预训练期间共享相同的目标函数，其中它们使用单向语言模型来学习通用的语言表示。

我们认为，现有技术限制了预训练表示的能力，尤其是在微调方法方面。其主要局限在于标准语言模型是单向的，这限制了预训练过程中可选用的架构。例如，在OpenAI GPT中，作者采用了从左到右的架构，使得Transformer自注意力层中的每个token仅能关注前序token（Vaswani et al., 2017）。此类限制对于句子级任务而言并非最优，当将基于微调的方法应用于问答等标记级任务时可能造成严重危害，因为在这些任务中同时融合双向上下文至关重要。

在本文中，我们提出BERT：以改进基于微调的方法**双向**的**编**码器**表**示自**转**换器。**BERT**通过使用受Cloze任务启发的“掩码语言模型”（MLM）预训练目标，缓解了前述的单向性约束：Taylor, 1953）。该掩码语言模型会随机对输入的Token进行掩码，其目标是预测被掩码

word based only on its context. Unlike left-to-right language model pre-training, the MLM objective enables the representation to fuse the left and the right context, which allows us to pre-train a deep bidirectional Transformer. In addition to the masked language model, we also use a “next sentence prediction” task that jointly pre-trains text-pair representations. The contributions of our paper are as follows:

- We demonstrate the importance of bidirectional pre-training for language representations. Unlike Radford et al. (2018), which uses unidirectional language models for pre-training, BERT uses masked language models to enable pre-trained deep bidirectional representations. This is also in contrast to Peters et al. (2018a), which uses a shallow concatenation of independently trained left-to-right and right-to-left LMs.
- We show that pre-trained representations reduce the need for many heavily-engineered task-specific architectures. BERT is the first fine-tuning based representation model that achieves state-of-the-art performance on a large suite of sentence-level and token-level tasks, outperforming many task-specific architectures.
- BERT advances the state of the art for eleven NLP tasks. The code and pre-trained models are available at <https://github.com/google-research/bert>.

2 Related Work

There is a long history of pre-training general language representations, and we briefly review the most widely-used approaches in this section.

2.1 Unsupervised Feature-based Approaches

Learning widely applicable representations of words has been an active area of research for decades, including non-neural (Brown et al., 1992; Ando and Zhang, 2005; Blitzer et al., 2006) and neural (Mikolov et al., 2013; Pennington et al., 2014) methods. Pre-trained word embeddings are an integral part of modern NLP systems, offering significant improvements over embeddings learned from scratch (Turian et al., 2010). To pre-train word embedding vectors, left-to-right language modeling objectives have been used (Mnih and Hinton, 2009), as well as objectives to discriminate correct from incorrect words in left and right context (Mikolov et al., 2013).

These approaches have been generalized to coarser granularities, such as sentence embeddings (Kiros et al., 2015; Logeswaran and Lee, 2018) or paragraph embeddings (Le and Mikolov, 2014). To train sentence representations, prior work has used objectives to rank candidate next sentences (Jernite et al., 2017; Logeswaran and Lee, 2018), left-to-right generation of next sentence words given a representation of the previous sentence (Kiros et al., 2015), or denoising auto-encoder derived objectives (Hill et al., 2016).

ELMo and its predecessor (Peters et al., 2017, 2018a) generalize traditional word embedding research along a different dimension. They extract *context-sensitive* features from a left-to-right and a right-to-left language model. The contextual representation of each token is the concatenation of the left-to-right and right-to-left representations. When integrating contextual word embeddings with existing task-specific architectures, ELMo advances the state of the art for several major NLP benchmarks (Peters et al., 2018a) including question answering (Rajpurkar et al., 2016), sentiment analysis (Socher et al., 2013), and named entity recognition (Tjong Kim Sang and De Meulder, 2003). Melamud et al. (2016) proposed learning contextual representations through a task to predict a single word from both left and right context using LSTMs. Similar to ELMo, their model is feature-based and not deeply bidirectional. Fedus et al. (2018) shows that the cloze task can be used to improve the robustness of text generation models.

2.2 Unsupervised Fine-tuning Approaches

As with the feature-based approaches, the first works in this direction only pre-trained word embedding parameters from unlabeled text (Collobert and Weston, 2008).

More recently, sentence or document encoders which produce contextual token representations have been pre-trained from unlabeled text and fine-tuned for a supervised downstream task (Dai and Le, 2015; Howard and Ruder, 2018; Radford et al., 2018). The advantage of these approaches is that few parameters need to be learned from scratch. At least partly due to this advantage, OpenAI GPT (Radford et al., 2018) achieved previously state-of-the-art results on many sentence-level tasks from the GLUE benchmark (Wang et al., 2018a). Left-to-right language model-

仅基于上下文来预测该词。不同于从左到-右语言模型预训练, MLM 目标使得表示能够融合左侧与右侧的上下文, 这使我们得以预-训练一个深度双向 Transformer。除掩码语言模型外, 我们还使用一项“下一句预测”任务, 用于联合预-训练文本对的表示。本文的贡献如下:

- 我们展示了双向预训练对于语言表征的重要性。与 Radford 等人 (2018)采用单向语言模型进行预训练不同, BERT使用掩码语言模型来构建预训练的深度双向表征。这也与 Peters 等人(2018a)形成对比, 后者仅使用了独立训练的从左到右和从右到左语言模型的浅层拼接。
- 我们证明, 预训练表征能够减少对众多高度定制化任务特定架构的依赖。BERT 是首个基于微调的表征模型, 在大量句子级和 词元级任务上均取得最先进水平, 其性能优于诸多任务特定的架构。

- BERT 在十一种 NLP 任务上将现有技术推向了新的高度。代码与预训练模型可在此获取: <https://github.com/google-research/bert>.

2 相关工作

通用语言表征的预训练拥有悠久的历史, 本节将简要回顾其中最广泛使用的方法。

2.1 无监督特征方法

学习广泛适用的词表示形式数十年来一直是活跃的研究领域, 包括非神经网络 (Brown 等人, 1992; Ando 与 Zhang, 2005; Blitzer 等人, 2006) 和神经网络 (Mikolov 等人, 2013; Pennington 等人, 2014) 方法。预训练的词嵌入已成为现代自然语言处理系统不可或缺的一部分, 相较于从零开始训练的嵌入带来了显著的提升 (Turian 等人, 2010)。为预训练词嵌入向量, 研究采用了从左到右的语言建模目标 (Mnih 和 Hinton, 2009), 以及旨在区分左右上下文中正确与错误词汇的目标 (Mikolov 等人, 2013)。

这些方法已被推广至更粗的粒度, 例如句子嵌入 (Kiros 等人, 2015; Logeswaran 和 Lee, 2018) 或段落嵌入 (Le 和 Mikolov, 2014)。为训练句子表示, 已有工作采用了旨在对候选下一句进行排序的目标 (Jernite 等人, 2017; Logeswaran 和 Lee, 2018), 给定前一句的表示后从左到右生成下一句词的生成目标 (Kiros 等人, 2015), 或源自去噪自编码器的衍生目标 (Hill 等人, 2016)。

ELMo及其前期工作 (Peters等, 2017, 2018a) 沿着不同的维度对传统词嵌入研究进行了推广。它们提取上下文敏感特征, 由从左至右和从右至左的语言模型提取。每个词的上下文表示为左右方向表示的拼接。当将上下文相关的词嵌入整合到现有的任务特定架构中时, ELMo在许多主要NLP基准测试中提升了最先进水平 (Peters等, 2018a) 包括问答 (Rajpurkar等, 2016)、情感分析 (Socher等, 2013) 和命名实体识别 (Tjong Kim Sang和De Meulder, 2003)。Melamud等 (2016) 提出利用LSTM学习上下文表示, 该方法通过预测左右上下文中单个单词的任务实现。与ELMo类似, 他们的模型是基于特征的, 并非深度双向的。Fedus等人 (2018) 表明, 完形填空任务可用于提升文本生成模型的鲁棒性。

2.2 无监督微调方法

与基于特征的方法类似, 该方向的首批工作仅从无标注文本中预训练词嵌入参数 (Collobert 和 Weston, 2008)。

最近, 生成上下文词元表示的句子或文档编码器已从无标签文本中进行预训练, 并为有监督的下游任务进行了微调 (Dai和Le, 2015; Howard和Ruder, 2018; Radford等人, 2018)。这些方法的优势在于需要从头学习的参数很少。至少部分归功于这一优势, OpenAI GPT (Radford等人, 2018在多项来自GLUE基准 (GLUE benchmark) 的句子级任务上取得了此前的最先进成果 (Wang等人, 2018a)。从左到右的语言模型-

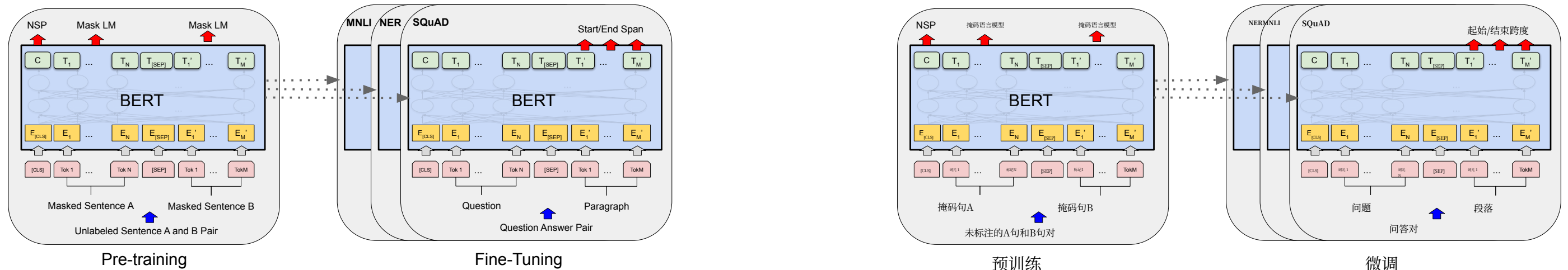


Figure 1: Overall pre-training and fine-tuning procedures for BERT. Apart from output layers, the same architectures are used in both pre-training and fine-tuning. The same pre-trained model parameters are used to initialize models for different down-stream tasks. During fine-tuning, all parameters are fine-tuned. [CLS] is a special symbol added in front of every input example, and [SEP] is a special separator token (e.g. separating questions/answers).

ing and auto-encoder objectives have been used for pre-training such models (Howard and Ruder, 2018; Radford et al., 2018; Dai and Le, 2015).

2.3 Transfer Learning from Supervised Data

There has also been work showing effective transfer from supervised tasks with large datasets, such as natural language inference (Conneau et al., 2017) and machine translation (McCann et al., 2017). Computer vision research has also demonstrated the importance of transfer learning from large pre-trained models, where an effective recipe is to fine-tune models pre-trained with ImageNet (Deng et al., 2009; Yosinski et al., 2014).

3 BERT

We introduce BERT and its detailed implementation in this section. There are two steps in our framework: *pre-training* and *fine-tuning*. During pre-training, the model is trained on unlabeled data over different pre-training tasks. For fine-tuning, the BERT model is first initialized with the pre-trained parameters, and all of the parameters are fine-tuned using labeled data from the downstream tasks. Each downstream task has separate fine-tuned models, even though they are initialized with the same pre-trained parameters. The question-answering example in Figure 1 will serve as a running example for this section.

A distinctive feature of BERT is its unified architecture across different tasks. There is mini-

mal difference between the pre-trained architecture and the final downstream architecture.

Model Architecture BERT’s model architecture is a multi-layer bidirectional Transformer encoder based on the original implementation described in Vaswani et al. (2017) and released in the `tensor2tensor` library.¹ Because the use of Transformers has become common and our implementation is almost identical to the original, we will omit an exhaustive background description of the model architecture and refer readers to Vaswani et al. (2017) as well as excellent guides such as “The Annotated Transformer.”²

In this work, we denote the number of layers (i.e., Transformer blocks) as L , the hidden size as H , and the number of self-attention heads as A .³ We primarily report results on two model sizes: **BERT_{BASE}** ($L=12$, $H=768$, $A=12$, Total Parameters=110M) and **BERT_{LARGE}** ($L=24$, $H=1024$, $A=16$, Total Parameters=340M).

BERT_{BASE} was chosen to have the same model size as OpenAI GPT for comparison purposes. Critically, however, the BERT Transformer uses bidirectional self-attention, while the GPT Transformer uses constrained self-attention where every token can only attend to context to its left.⁴

¹<https://github.com/tensorflow/tensor2tensor>

²<http://nlp.seas.harvard.edu/2018/04/03/attention.html>

³In all cases we set the feed-forward/filter size to be $4H$, i.e., 3072 for the $H = 768$ and 4096 for the $H = 1024$.

⁴We note that in the literature the bidirectional Trans-

自回归与自动编码器目标已被用于此类模型的预训练 (Howard和Ruder, 2018; Radford等人, 2018; Dai和Le, 2015)。

2.3 基于监督数据的迁移学习

也有研究展示了从带有大规模数据集的监督任务中进行有效迁移的可行性，例如自然语言推理 (Conneau 等人, 2017) 以及机器翻译 (McCann 等人, 2017)。计算机视觉研究也证实了从大型预训练模型进行迁移学习的重要性，其中一种高效的策略是微调在 ImageNet 上预训练的模型 (Deng 等人, 2009; Yosinski 等人, 2014)。

3 BERT

本节将介绍 BERT 及其详细实现。我们的框架包含两个步骤：预训练和微调。在预训练阶段，模型会在不同预训练任务上使用无标签数据进行训练。对于微调阶段，BERT 模型首先基于预训练参数进行初始化，所有参数均使用下游任务的标注数据进行微调。尽管它们都使用相同的预训练参数进行初始化，但每个下游任务仍有独立的微调模型。图 1 中的问答示例将作为本节贯穿始终的示例。

BERT的一个显著特征是其针对不同任务的统一架构。存在极小的

预训练架构与最终下游架构之间的差异。

模型架构 BERT 的模型架构是一个多层双向 Transformer 编码器，基于 Vaswani 等人 (2017) 提出的原始实现，并发布于 `tensor2tensor` 库中。¹ 由于 Transformer 的使用已十分普遍，且我们的实现与原版几乎完全相同，因此我们将省略对模型架构详尽的背景描述，并将读者引荐至 Vaswani 等人 (2017)，以及诸如《带注释的 Transformer》等优秀指南。²

在本工作中，我们将层数（即 Transformer 模块）的数量记为 L ，隐藏层大小记为 H ，自注意力头的数量记为 A 。³ 我们主要报告两种模型规模的实验结果：BERT_{BASE} ($L=12$, $H=768$, $A=12$, 总参数量=110M) 与 BERT_{LARGE} ($L=24$, $H=1024$, $A=16$, 总参数量=340M)。

出于比较目的，所选取的BERT_{BASE}在模型规模上与OpenAI GPT保持一致。然而关键的区别在于，BERT Transformer采用双向自注意力机制，而GPT Transformer采用受限的自注意力机制，在该机制中每个token只能关注其左侧的上下文信息。⁴

¹<https://github.com/tensorflow/tensor2tensor>

²<http://nlp.seas.harvard.edu/2018/04/03/attention.html>³在所有情况下，我们将前馈/滤波器尺寸设置为 $4H$ ，即 $H = 768$ 对应 3072， $H = 1024$ 对应 4096。⁴需要指出的是，在现有文献中，双向 Trans-

Input/Output Representations To make BERT handle a variety of down-stream tasks, our input representation is able to unambiguously represent both a single sentence and a pair of sentences (e.g., $\langle \text{Question, Answer} \rangle$) in one token sequence. Throughout this work, a “sentence” can be an arbitrary span of contiguous text, rather than an actual linguistic sentence. A “sequence” refers to the input token sequence to BERT, which may be a single sentence or two sentences packed together.

We use WordPiece embeddings (Wu et al., 2016) with a 30,000 token vocabulary. The first token of every sequence is always a special classification token ($[\text{CLS}]$). The final hidden state corresponding to this token is used as the aggregate sequence representation for classification tasks. Sentence pairs are packed together into a single sequence. We differentiate the sentences in two ways. First, we separate them with a special token ($[\text{SEP}]$). Second, we add a learned embedding to every token indicating whether it belongs to sentence A or sentence B. As shown in Figure 1, we denote input embedding as E , the final hidden vector of the special $[\text{CLS}]$ token as $C \in \mathbb{R}^H$, and the final hidden vector for the i^{th} input token as $T_i \in \mathbb{R}^H$.

For a given token, its input representation is constructed by summing the corresponding token, segment, and position embeddings. A visualization of this construction can be seen in Figure 2.

3.1 Pre-training BERT

Unlike Peters et al. (2018a) and Radford et al. (2018), we do not use traditional left-to-right or right-to-left language models to pre-train BERT. Instead, we pre-train BERT using two unsupervised tasks, described in this section. This step is presented in the left part of Figure 1.

Task #1: Masked LM Intuitively, it is reasonable to believe that a deep bidirectional model is strictly more powerful than either a left-to-right model or the shallow concatenation of a left-to-right and a right-to-left model. Unfortunately, standard conditional language models can only be trained left-to-right *or* right-to-left, since bidirectional conditioning would allow each word to indirectly “see itself”, and the model could trivially predict the target word in a multi-layered context.

former is often referred to as a “Transformer encoder” while the left-context-only version is referred to as a “Transformer decoder” since it can be used for text generation.

In order to train a deep bidirectional representation, we simply mask some percentage of the input tokens at random, and then predict those masked tokens. We refer to this procedure as a “masked LM” (MLM), although it is often referred to as a *Cloze* task in the literature (Taylor, 1953). In this case, the final hidden vectors corresponding to the mask tokens are fed into an output softmax over the vocabulary, as in a standard LM. In all of our experiments, we mask 15% of all WordPiece tokens in each sequence at random. In contrast to denoising auto-encoders (Vincent et al., 2008), we only predict the masked words rather than reconstructing the entire input.

Although this allows us to obtain a bidirectional pre-trained model, a downside is that we are creating a mismatch between pre-training and fine-tuning, since the $[\text{MASK}]$ token does not appear during fine-tuning. To mitigate this, we do not always replace “masked” words with the actual $[\text{MASK}]$ token. The training data generator chooses 15% of the token positions at random for prediction. If the i -th token is chosen, we replace the i -th token with (1) the $[\text{MASK}]$ token 80% of the time (2) a random token 10% of the time (3) the unchanged i -th token 10% of the time. Then, T_i will be used to predict the original token with cross entropy loss. We compare variations of this procedure in Appendix C.2.

Task #2: Next Sentence Prediction (NSP)

Many important downstream tasks such as Question Answering (QA) and Natural Language Inference (NLI) are based on understanding the *relationship* between two sentences, which is not directly captured by language modeling. In order to train a model that understands sentence relationships, we pre-train for a binarized *next sentence prediction* task that can be trivially generated from any monolingual corpus. Specifically, when choosing the sentences A and B for each pre-training example, 50% of the time B is the actual next sentence that follows A (labeled as IsNext), and 50% of the time it is a random sentence from the corpus (labeled as NotNext). As we show in Figure 1, C is used for next sentence prediction (NSP).⁵ Despite its simplicity, we demonstrate in Section 5.1 that pre-training towards this task is very beneficial to both QA and NLI.⁶

⁵The final model achieves 97%-98% accuracy on NSP.

⁶The vector C is not a meaningful sentence representation without fine-tuning, since it was trained with NSP.

输入/输出表示 为了使BERT能够处理各类下游任务，我们的输入表示能够在单一的词元序列中无歧义地表示单句或双句（例如， $\langle \text{问题, 回答} \rangle$ ）。在本研究的全篇中，“句子”指代的是任意连续的文本跨度，而非实际的语言学句子。“序列”则是指输入至BERT的词元序列，它可能是一个单独的句子，也可能是将两个句子拼接在一起的组合。

我们采用 WordPiece 词嵌入 (Wu 等人, 2016)，词表大小为 30,000。每个序列的首个词元始终是一个特殊的分类标记 ($[\text{CLS}]$)。对应于此标记的最终隐藏状态被用作分类任务的聚合序列表示。句子对会被打包为单一序列。我们通过两种方式区分这些句子。首先，我们使用特殊标记 ($[\text{SEP}]$)。其次，我们为每个词元添加一个可学习的嵌入向量，以表明它属于句子 A 还是句子 B。如图1所示，我们将输入嵌入记为 E ，将特殊 $[\text{CLS}]$ 标记的最终隐藏向量记为 $C \in \mathbb{R}^H$ ，并将 i^{th} 输入词元 $T_i \in \mathbb{R}^H$ 的最终隐藏向量记为。

对于给定词元，其输入表示由对应的词元、段和位置嵌入向量相加得到。该构建过程的可视化效果如图 2 所示。

3.1 BERT的预训练

不同于 Peters 等人 (2018a) 与 Radford 等人 (2018)，我们并未使用传统的从左到右或从右到左语言模型来预训练 BERT。相反，我们采用本节所述的两项无监督任务来预训练 BERT。该步骤如图 1 左侧部分所示。

任务#1: 掩码语言模型 直观上，人们有理由相信，深度双向模型的表达能力严格优于仅从左到右的模型，也优于从左到右与从右到左模型的浅层拼接。然而，标准的条件语言模型只能从左到右或从右到左进行训练，因为若采用双向条件约束，每个词都会间接地“看到”自身，模型便能轻易在多层语境中预测出目标词。

前者通常被称为“Transformer 编码器”，而仅包含左上下文的版本则被称为“Transformer 解码器”，因为它可用于文本生成。

为了训练深度双向表示，我们只需随机遮盖输入词元的一定比例，并预测这些被遮盖的词元。我们将此过程称为“掩码语言模型” (MLM)，尽管在文献中它常被称为一种完形填空任务 (Taylor, 1953)。在此情况下，对应于掩码词元的最终隐藏向量将被送入覆盖整个词汇表的输出 Softmax 层，正如标准语言模型所做的那样。在我们所有的实验中，对每个序列中随机的 15% 的 WordPiece 词元进行遮盖。与去噪自编码器 (Vincent et al., 2008) 不同，我们仅预测被遮盖的词，而非重建整个输入。

尽管这使我们能够获得一个双向预训练模型，但缺点是我们在预训练与微调之间引入了不匹配，因为 $[\text{MASK}]$ 令牌在微调期间不出现。为缓解这一问题，我们并不总是将“被掩码”的词替换为实际的 $[\text{MASK}]$ 令牌。训练数据生成器会随机选择 15% 的令牌位置进行预测。如果选中了第 i 个令牌，我们会替换第 i 个令牌为 (1) 80% 的情况下用 $[\text{MASK}]$ 令牌， (2) 10% 的情况下用一个随机令牌， (3) 10% 的情况下保持第 i 个令牌不变。之后， T_i 将使用交叉熵损失来预测原始令牌。我们在附录 C.2 中对这一过程的变体进行了比较。

任务 #2: 下一句预测 (NSP) 许多重要的下游任务，如问答 (QA) 和自然语言推理 (NLI)，都基于对关系两个句子之间的关系，而这无法通过语言建模直接捕获。为了训练能够理解句子间关系的模型，我们针对二值化的下一句预测任务进行了预训练，该任务可从任意单语语料库中轻松生成。具体而言，在为每个预训练示例选取句子时 A 和 B，有 50% 的概率 B 是实际的紧邻 A 之后的真实下一句 (标记为 IsNext)，而在另外 50% 的情况下，它来自语料库中的随机句子 (标记为 NotNext)。如图所示1， C 被用于下一句预测 (NSP)。⁵ 尽管其设计简单，我们在第 5.1 节中证明，针对该任务的预训练对 QA 和 NLI 均大有裨益。⁶

⁵最终模型在 NSP 任务上的准确率达到 97%-98%。⁶由于该向量是基于 NSP 目标训练的，因此在未经微调的情况下， C 无法构成有意义的句子表示。

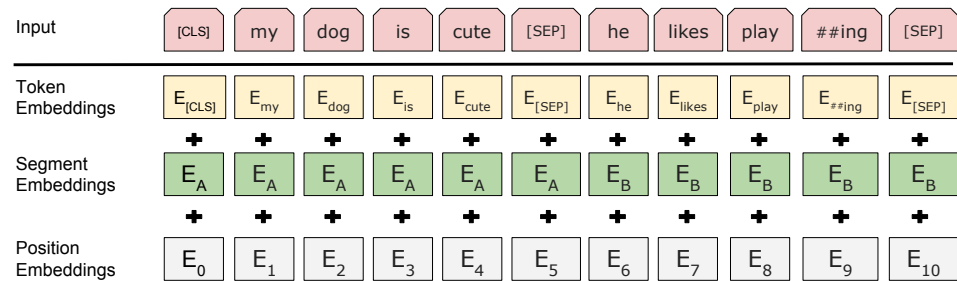


Figure 2: BERT input representation. The input embeddings are the sum of the token embeddings, the segmentation embeddings and the position embeddings.

The NSP task is closely related to representation-learning objectives used in Jernite et al. (2017) and Logeswaran and Lee (2018). However, in prior work, only sentence embeddings are transferred to down-stream tasks, where BERT transfers all parameters to initialize end-task model parameters.

Pre-training data The pre-training procedure largely follows the existing literature on language model pre-training. For the pre-training corpus we use the BooksCorpus (800M words) (Zhu et al., 2015) and English Wikipedia (2,500M words). For Wikipedia we extract only the text passages and ignore lists, tables, and headers. It is critical to use a document-level corpus rather than a shuffled sentence-level corpus such as the Billion Word Benchmark (Chelba et al., 2013) in order to extract long contiguous sequences.

3.2 Fine-tuning BERT

Fine-tuning is straightforward since the self-attention mechanism in the Transformer allows BERT to model many downstream tasks—whether they involve single text or text pairs—by swapping out the appropriate inputs and outputs. For applications involving text pairs, a common pattern is to independently encode text pairs before applying bidirectional cross attention, such as Parikh et al. (2016); Seo et al. (2017). BERT instead uses the self-attention mechanism to unify these two stages, as encoding a concatenated text pair with self-attention effectively includes *bidirectional* cross attention between two sentences.

For each task, we simply plug in the task-specific inputs and outputs into BERT and fine-tune all the parameters end-to-end. At the input, sentence A and sentence B from pre-training are analogous to (1) sentence pairs in paraphrasing, (2) hypothesis-premise pairs in entailment, (3) question-passage pairs in question answering, and

(4) a degenerate text- $\emptyset$ pair in text classification or sequence tagging. At the output, the token representations are fed into an output layer for token-level tasks, such as sequence tagging or question answering, and the [CLS] representation is fed into an output layer for classification, such as entailment or sentiment analysis.

Compared to pre-training, fine-tuning is relatively inexpensive. All of the results in the paper can be replicated in at most 1 hour on a single Cloud TPU, or a few hours on a GPU, starting from the exact same pre-trained model.⁷ We describe the task-specific details in the corresponding subsections of Section 4. More details can be found in Appendix A.5.

4 Experiments

In this section, we present BERT fine-tuning results on 11 NLP tasks.

4.1 GLUE

The General Language Understanding Evaluation (GLUE) benchmark (Wang et al., 2018a) is a collection of diverse natural language understanding tasks. Detailed descriptions of GLUE datasets are included in Appendix B.1.

To fine-tune on GLUE, we represent the input sequence (for single sentence or sentence pairs) as described in Section 3, and use the final hidden vector $C \in \mathbb{R}^H$ corresponding to the first input token ([CLS]) as the aggregate representation. The only new parameters introduced during fine-tuning are classification layer weights $W \in \mathbb{R}^{K \times H}$, where K is the number of labels. We compute a standard classification loss with C and W , i.e., $\log(\text{softmax}(CW^T))$.

⁷For example, the BERT SQuAD model can be trained in around 30 minutes on a single Cloud TPU to achieve a Dev F1 score of 91.0%.

⁸See (10) in <https://gluebenchmark.com/faq>.

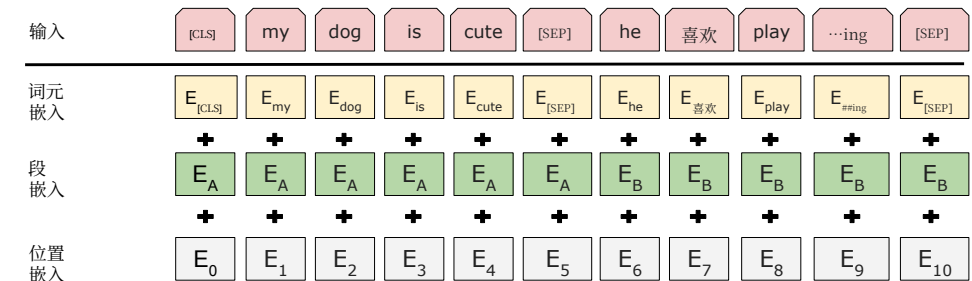


图2: BERT输入表示。输入嵌入为Token嵌入、段嵌入与位置嵌入之和。

NSP任务与表征学习目标密切相关，这些目标被应用于 Jernite等 (2017) 和Logeswaran和Lee (2018)。然而，在以往的研究中，仅将句子嵌入迁移至下游任务，而BERT则迁移所有参数以初始化目标任务的模型参数。

预训练数据 预训练过程大体遵循语言模型预训练的现有文献。对于预训练语料库，我们使用BooksCorpus (8亿词) (Zhu等, 2015) 以及英文维基百科 (25亿词)。对于维基百科，我们仅提取文本段落，忽略列表、表格和标题。为了提取长的连续序列，使用文档级语料库而非像十亿词基准 (Chelba等, 2013) 那样打乱的句子级语料库至关重要。

3.2 微调BERT

微调十分直接，因为Transformer中的自注意力机制使BERT能够通过更换相应的输入和输出来处理多种下游任务——无论是处理单段文本还是文本对。对于涉及文本对的应用，常见的做法是先独立编码文本对，再应用双向交叉注意力，例如Parikh等人(2016); Seo等人(2017)。BERT该模型改用自注意力机制来统一这两个阶段，因为在自注意力机制下对拼接后的文本对进行编码，实际上相当于包含了双向交叉注意力。

对于每个任务，我们只需将特定任务的输入和输出直接接入BERT，并以端到端的方式微调所有参数。在输入端，预训练中的句子A与句子B可类比为：(1) 释义任务中的句子对，(2) 蕴含任务中的假设-前提对，(3) 问答任务中的问题-段落对，以及

(4) 文本分类或序列标注任务中退化至单个文本- $\emptyset$ 对的情形。在输出端，token表示会被送入输出层以处理 token 级任务（如序列标注或问答），而 [CLS] 表示则会被送入用于分类任务（如蕴含判断或情感分析）的输出层。

与预训练相比，微调成本相对较低。本文的所有结果均可在单个 Cloud TPU 上至多 1 小时内，或在 GPU 上数小时内复现，只需从完全相同的预训练模型开始即可。⁷ 我们在第 4 节的相应小节中描述任务的具体细节。更多细节详见附录 A.5。

4 实验

在本节中，我们展示 BERT 在 11 项自然语言处理 (NLP) 任务上的微调结果。

4.1 GLUE

通用语言理解评估 (GLUE) 基准 (Wang et al., 2018a) 包含一系列多样化的自然语言理解任务。GLUE 数据集的详细描述收录于附录 B.1。

在GLUE上进行微调时，我们按照第3节所述来表示输入序列（针对单句或句子对），并使用 $C \in \mathbb{R}^H$ 对应于第一个输入标记 ([CLS]) 的最终隐藏向量作为聚合表示。微调过程中引入的新参数仅为分类层权重 $W \in \mathbb{R}^{K \times H}$ ，其中 K 为标签数量。我们使用 C 和 W 计算标准分类损失，即： $\log(\text{softmax}(CW^T))$ 。

⁷例如，在单个 Cloud TPU 上训练 BERT SQuAD 模型仅需约 30 分钟，即可达到 91.0% 的开发集 (Dev) F1 分数。⁸ 详见 <https://gluebenchmark.com/faq> 中的说明 (10)。

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	Average -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

Table 1: GLUE Test results, scored by the evaluation server (<https://gluebenchmark.com/leaderboard>). The number below each task denotes the number of training examples. The ‘‘Average’’ column is slightly different than the official GLUE score, since we exclude the problematic WNLI set.⁸ BERT and OpenAI GPT are single-model, single task. F1 scores are reported for QQP and MRPC, Spearman correlations are reported for STS-B, and accuracy scores are reported for the other tasks. We exclude entries that use BERT as one of their components.

We use a batch size of 32 and fine-tune for 3 epochs over the data for all GLUE tasks. For each task, we selected the best fine-tuning learning rate (among 5e-5, 4e-5, 3e-5, and 2e-5) on the Dev set. Additionally, for BERT_{LARGE} we found that fine-tuning was sometimes unstable on small datasets, so we ran several random restarts and selected the best model on the Dev set. With random restarts, we use the same pre-trained checkpoint but perform different fine-tuning data shuffling and classifier layer initialization.⁹

Results are presented in Table 1. Both BERT_{BASE} and BERT_{LARGE} outperform all systems on all tasks by a substantial margin, obtaining 4.5% and 7.0% respective average accuracy improvement over the prior state of the art. Note that BERT_{BASE} and OpenAI GPT are nearly identical in terms of model architecture apart from the attention masking. For the largest and most widely reported GLUE task, MNLI, BERT obtains a 4.6% absolute accuracy improvement. On the official GLUE leaderboard¹⁰, BERT_{LARGE} obtains a score of 80.5, compared to OpenAI GPT, which obtains 72.8 as of the date of writing.

We find that BERT_{LARGE} significantly outperforms BERT_{BASE} across all tasks, especially those with very little training data. The effect of model size is explored more thoroughly in Section 5.2.

4.2 SQuAD v1.1

The Stanford Question Answering Dataset (SQuAD v1.1) is a collection of 100k crowd-sourced question/answer pairs (Rajpurkar et al., 2016). Given a question and a passage from

⁹The GLUE data set distribution does not include the Test labels, and we only made a single GLUE evaluation server submission for each of BERT_{BASE} and BERT_{LARGE}.

¹⁰<https://gluebenchmark.com/leaderboard>

Wikipedia containing the answer, the task is to predict the answer text span in the passage.

As shown in Figure 1, in the question answering task, we represent the input question and passage as a single packed sequence, with the question using the A embedding and the passage using the B embedding. We only introduce a start vector $S \in \mathbb{R}^H$ and an end vector $E \in \mathbb{R}^H$ during fine-tuning. The probability of word i being the start of the answer span is computed as a dot product between T_i and S followed by a softmax over all of the words in the paragraph: $P_i = \frac{e^{S \cdot T_i}}{\sum_j e^{S \cdot T_j}}$. The analogous formula is used for the end of the answer span. The score of a candidate span from position i to position j is defined as $S \cdot T_i + E \cdot T_j$, and the maximum scoring span where $j \geq i$ is used as a prediction. The training objective is the sum of the log-likelihoods of the correct start and end positions. We fine-tune for 3 epochs with a learning rate of 5e-5 and a batch size of 32.

Table 2 shows top leaderboard entries as well as results from top published systems (Seo et al., 2017; Clark and Gardner, 2018; Peters et al., 2018a; Hu et al., 2018). The top results from the SQuAD leaderboard do not have up-to-date public system descriptions available,¹¹ and are allowed to use any public data when training their systems. We therefore use modest data augmentation in our system by first fine-tuning on TriviaQA (Joshi et al., 2017) before fine-tuning on SQuAD.

Our best performing system outperforms the top leaderboard system by +1.5 F1 in ensembling and +1.3 F1 as a single system. In fact, our single BERT model outperforms the top ensemble system in terms of F1 score. Without TriviaQA fine-

¹¹QANet is described in Yu et al. (2018), but the system has improved substantially after publication.

系统	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	平均 -
OpenAI 之前的 SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
BERT _{BASE}	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
BERT _{LARGE}	86.7/85.9	72.1	92.7	94.9	60.5	86.5	89.3	70.1	82.1

表 1: GLUE 测试集结果, 由评估服务器 (<https://gluebenchmark.com/leaderboard>) 评分。各任务下方的数字表示该任务的训练样本数量。‘‘平均’’列的数值与官方 GLUE 得分略有差异, 因为我们未纳入存在问题的 WNLI 数据集。⁸ BERT 与 OpenAI GPT 属于单模型、单任务架构。QQP 和 MRPC 采用 F1 分数进行评估, STS-B 采用 Spearman 相关系数进行评估, 其余任务则采用准确率进行评估。我们已剔除所有将 BERT 作为其组件之一的参赛条目。

我们将批大小设置为32, 对所有GLUE任务的数据进行3个epoch的微调。对于每个任务, 我们在Dev集上选出最佳微调学习率(候选值为5e-5、4e-5、3e-5和2e-5)。此外, 我们发现对于 BERT_{LARGE} 在小数据集上的微调有时不稳定, 因此我们执行了多次随机重启, 并选择了在Dev集上效果最佳的模型。在随机重启过程中, 我们使用相同的预训练检查点, 但采用了不同的微调数据打乱方式以及分类器层的初始化方法。⁹

结果如表1。两者BERT_{BASE} 与 BERT_{LARGE} 在所有任务上均以显著优势超越所有系统, 取得4.5% 和 7.0% 相较于此前最优水平的相对平均准确率提升。需注意BERT_{BASE} 与 OpenAI GPT 在模型架构上几乎完全一致仅注意力掩码机制不同。针对规模最大且报道最广的GLUE 任务 MNLI, BERT 实现了 4.6% 的绝对准确率提升。在官方GLUE 排行榜¹⁰, BERT_{LARGE} 取得分为 80.5, 相比之下 OpenAI GPT 取得72.8 (截至撰写日期)。

我们发现 BERT_{LARGE} 在各项任务中均显著优于 BERT_{BASE}, 尤其是在训练数据极少的任务上。模型规模的影响将在第 5.2 节中进行更深入的探讨。

4.2 SQuAD v1.1

斯坦福问题回答数据集 (SQuAD v1.1) 是一个包含 10 万个众包问答对的数据集 (Rajpurkar 等人, 2016)。给定一个问题和一个来自……的段落,

⁹GLUE 数据集的划分不包含测试集标签, 且我们仅为 BERT_{BASE} 和 BERT_{LARGE} 分别在 GLUE 评估服务器上提交了一次评测结果。¹⁰<https://gluebenchmark.com/leaderboard>

维基百科中包含答案的段落, 该任务的目的是预测该段落中的答案文本片段。

如图1, 在问答任务中, 我们将输入的问题与段落表示为一个单一的拼接序列, 其中问题使用A 嵌入, 而段落使用B 嵌入。我们仅在微调阶段引入起始向量 $S \in \mathbb{R}^H$ 以及一个结束向量 $E \in \mathbb{R}^H$ 在微调阶段。该词 i 成为答案跨度起始词的概率, 计算为它与 T_i 和 S 之间的点积, 并对段落中的所有词进行 softmax归一化得到: $P_i = \frac{e^{S \cdot T_i}}{\sum_j e^{S \cdot T_j}}$ 。类似的公式也用于确定答案跨度的结束位置。从位置 i 到位置 j 的候选跨度得分定义为 $S \cdot T_i + E \cdot T_j$, 并选用得分最高的跨度 (使用 $j \geq i$) 作为预测结果。训练目标是正确答案起止位置对数似然值之和。我们以学习率 5e-5和批次大小32微调3个epoch。

表 2 展示了排行榜前列的记录, 以及已发表顶尖系统的结果 (Seo 等。 ,2017 ;Clark 与 Gardner , 2018;Peters 等。 ,2018a ;Hu 等。 , 2018)。SQuAD排行榜上的顶尖成绩目前缺乏最新的公开¹¹ 系统说明, 且允许在训练系统时使用任何公开数据。因此, 我们的系统采用适度的数据增强策略, 首先在TriviaQA (Joshi等。 , 2017) 上进行微调, 随后在SQuAD上微调。

我们性能最优的系统在集成模式下比排行榜第一名的系统高出 +1.5 的 F1 分数, 在单模型模式下高出 +1.3 的 F1 分数。事实上, 我们独立的 BERT 模型在 F1 分数上已超越最佳集成系统。在没有 TriviaQA 微调-

¹¹QANet由Yu等人(2018) , 但该系统在发表后已得到显著改进。

System	Dev		Test	
	EM	F1	EM	F1
Top Leaderboard Systems (Dec 10th, 2018)				
Human	-	-	82.3	91.2
#1 Ensemble - nlnet	-	-	86.0	91.7
#2 Ensemble - QANet	-	-	84.5	90.5
Published				
BiDAF+ELMo (Single)	-	85.6	-	85.8
R.M. Reader (Ensemble)	81.2	87.9	82.3	88.5
Ours				
BERT _{BASE} (Single)	80.8	88.5	-	-
BERT _{LARGE} (Single)	84.1	90.9	-	-
BERT _{LARGE} (Ensemble)	85.8	91.8	-	-
BERT _{LARGE} (Sgl.+TriviaQA)	84.2	91.1	85.1	91.8
BERT _{LARGE} (Ens.+TriviaQA)	86.2	92.2	87.4	93.2

Table 2: SQuAD 1.1 results. The BERT ensemble is 7x systems which use different pre-training checkpoints and fine-tuning seeds.

System	Dev		Test	
	EM	F1	EM	F1
Top Leaderboard Systems (Dec 10th, 2018)				
Human	86.3	89.0	86.9	89.5
#1 Single - MIR-MRC (F-Net)	-	-	74.8	78.0
#2 Single - nlnet	-	-	74.2	77.1
Published				
unet (Ensemble)	-	-	71.4	74.9
SLQA+ (Single)	-		71.4	74.4
Ours				
BERT _{LARGE} (Single)	78.7	81.9	80.0	83.1

Table 3: SQuAD 2.0 results. We exclude entries that use BERT as one of their components.

tuning data, we only lose 0.1-0.4 F1, still outperforming all existing systems by a wide margin.¹²

4.3 SQuAD v2.0

The SQuAD 2.0 task extends the SQuAD 1.1 problem definition by allowing for the possibility that no short answer exists in the provided paragraph, making the problem more realistic.

We use a simple approach to extend the SQuAD v1.1 BERT model for this task. We treat questions that do not have an answer as having an answer span with start and end at the [CLS] token. The probability space for the start and end answer span positions is extended to include the position of the [CLS] token. For prediction, we compare the score of the no-answer span: $s_{\text{null}} = S \cdot C + E \cdot C$ to the score of the best non-null span

System	Dev	Test
ESIM+GloVe	51.9	52.7
ESIM+ELMo	59.1	59.2
OpenAI GPT	-	78.0
BERT _{BASE}	81.6	-
BERT _{LARGE}	86.6	86.3
Human (expert) [†]	-	85.0
Human (5 annotations) [†]	-	88.0

Table 4: SWAG Dev and Test accuracies. [†]Human performance is measured with 100 samples, as reported in the SWAG paper.

$\hat{s}_{i,j} = \max_{j \geq i} S \cdot T_i + E \cdot T_j$. We predict a non-null answer when $\hat{s}_{i,j} > s_{\text{null}} + \tau$, where the threshold τ is selected on the dev set to maximize F1. We did not use TriviaQA data for this model. We fine-tuned for 2 epochs with a learning rate of 5e-5 and a batch size of 48.

The results compared to prior leaderboard entries and top published work (Sun et al., 2018; Wang et al., 2018b) are shown in Table 3, excluding systems that use BERT as one of their components. We observe a +5.1 F1 improvement over the previous best system.

4.4 SWAG

The Situations With Adversarial Generations (SWAG) dataset contains 113k sentence-pair completion examples that evaluate grounded common-sense inference (Zellers et al., 2018). Given a sentence, the task is to choose the most plausible continuation among four choices.

When fine-tuning on the SWAG dataset, we construct four input sequences, each containing the concatenation of the given sentence (sentence A) and a possible continuation (sentence B). The only task-specific parameters introduced is a vector whose dot product with the [CLS] token representation C denotes a score for each choice which is normalized with a softmax layer.

We fine-tune the model for 3 epochs with a learning rate of 2e-5 and a batch size of 16. Results are presented in Table 4. BERT_{LARGE} outperforms the authors’ baseline ESIM+ELMo system by +27.1% and OpenAI GPT by 8.3%.

5 Ablation Studies

In this section, we perform ablation experiments over a number of facets of BERT in order to better understand their relative importance. Additional

系统	Dev		Test	
	EM	F1	EM	F1
顶级排行榜系统（2018年12月10日）				
人工	-	-	82.3	91.2
#1 集成模型 - nlnet	-	-	86.0	91.7
#2 集成 - QANot	-	-	84.5	90.5
已发表				
BiDAF+ELMo（单模型）	-	85.6	-	85.8
R.M. Reader（集成）	81.2	87.9	82.3	88.5
Ours				
BERT _{BASE} （单模型）	80.8	88.5	-	-
BERT _{LARGE} （单模型）	84.1	90.9	-	-
BERT _{LARGE} （集成）	85.8	91.8	-	-
BERT _{LARGE} （单模型+TriviaQA）	84.2	91.1	85.1	91.8
BERT _{LARGE} （集成模型+TriviaQA）	86.2	92.2	87.4	93.2

表2：SQuAD 1.1结果。BERT集成由7个系统组成，这些系统使用了不同的预训练检查点和微调种子。

系统	Dev		Test	
	EM	F1	EM	F1
排行榜前列系统（2018年12月10日）				
人工	86.3	89.0	86.9	89.5
#1 单设 - MIR-MRC (F-Net)	-	-	74.8	78.0
#2 单设 - nlnet	-	-	74.2	77.1
已发布				
unet (集成)	-	-	71.4	74.9
SLQA+ (单模型)	-		71.4	74.4
Ours				
BERT _{LARGE} (单模型)	78.7	81.9	80.0	83.1

表3: SQuAD 2.0结果。我们排除了将BERT作为组成部分之一的条目。

数据的情况下，我们仅损失 0.1-0.4 的 F1 分数，仍大幅领先于所有现有系统，达到 ¹² 的成绩。

4.3 SQuAD v2.0

SQuAD 2.0任务扩展了SQuAD 1.1的问题定义，通过允许所提供段落中不存在简短答案的情况，使该问题更加贴近实际。

我们采用一种简单的方法来扩展 SQuAD v1.1 BERT 模型以完成此任务。对于没有答案的问题，我们将其视为存在一个起始与终止位置均位于 [CLS] token 处的答案跨度。起始和终止答案跨度位置的概率空间已扩展，纳入了 [CLS] token 的位置。在预测时，我们将无答案跨度的得分： $s_{\text{null}} = S \cdot C + E \cdot C$ 与最优非空答案跨度的得分进行对比。

¹²我们使用的TriviaQA数据由TriviaQA-Wiki的段落组成，这些段落提取自文档的前400个词元，且至少包含一个提供的可能答案。

系统	开发测试	
ESIM+GloVe	51.9	52.7
ESIM+ELMo	59.1	59.2
OpenAI GPT	-	78.0
BERT _{BASE}	81.6	-
BERT _{LARGE}	86.6	86.3
人工（专家） [†]	-	85.0
人工（5个标注） [†]	-	88.0

表4: SWAG开发集与测试集准确率。 [†]人工表现基于100个样本进行测量，如SWAG论文中所述。

$\hat{s}_{i,j} = \max_{j \geq i} S \cdot T_i + E \cdot T_{j_0}$ 当 $\hat{s}_{i,j} > s_{\text{null}} + \tau$ 时，我们预测为非空答案，其中阈值 τ 在开发集上选定以最大化 F1 值。本模型未使用 TriviaQA 数据。我们以 5e-5 的学习率和 48 的批次大小对其进行了 2 个 epoch 的微调。

与先前列榜结果及顶级已发表论文（Sun等, 2018; Wang等, 2018b）相比的结果如表3所示，其中排除了将BERT作为组成部分之一的系统。我们观察到相较于之前的最佳系统，获得了 +5.1的F1性能提升。

4.4 SWAG

对抗性生成情境（SWAG）数据集包含11.3万个句子对补全示例，用于评估基于情境的常识推理能力（Zellers等, 2018）。给定一个句子，任务是从四个选项中选出最合理的接续内容。

在SWAG数据集上进行微调时，我们构建四个输入序列，每个序列包含给定句子（句子A）与可能续写（句子B）的拼接。引入的唯一任务特定参数是一个向量，该向量与 [CLS] token 表示的点积 C 代表每个选项的得分，并通过 softmax 层进行归一化。

我们在 3 个 epoch 中对模型进行微调，学习率为 2e-5，批次大小为 16。结果展示在表 4 所示。BERT_{LARGE} 优于作者的基线 ESIM+和 ELMo 系统，分别高出 +27.1% 和 OpenAI GPT 8.3%。

5 消融实验

在本节中，我们对BERT的多个方面进行了消融实验以更好地了解它们各自的相对重要性。补充的

¹²The TriviaQA data we used consists of paragraphs from TriviaQA-Wiki formed of the first 400 tokens in documents, that contain at least one of the provided possible answers.

Tasks	Dev Set				
	MNLI-m (Acc)	QNLI (Acc)	MRPC (Acc)	SST-2 (Acc)	SQuAD (F1)
BERT _{BASE}	84.4	88.4	86.7	92.7	88.5
No NSP	83.9	84.9	86.5	92.6	87.9
LTR & No NSP	82.1	84.3	77.5	92.1	77.8
+ BiLSTM	82.1	84.1	75.7	91.6	84.9

Table 5: Ablation over the pre-training tasks using the BERT_{BASE} architecture. “No NSP” is trained without the next sentence prediction task. “LTR & No NSP” is trained as a left-to-right LM without the next sentence prediction, like OpenAI GPT. “+ BiLSTM” adds a randomly initialized BiLSTM on top of the “LTR + No NSP” model during fine-tuning.

ablation studies can be found in Appendix C.

5.1 Effect of Pre-training Tasks

We demonstrate the importance of the deep bidirectionality of BERT by evaluating two pre-training objectives using exactly the same pre-training data, fine-tuning scheme, and hyperparameters as BERT_{BASE}:

No NSP: A bidirectional model which is trained using the “masked LM” (MLM) but without the “next sentence prediction” (NSP) task.

LTR & No NSP: A left-context-only model which is trained using a standard Left-to-Right (LTR) LM, rather than an MLM. The left-only constraint was also applied at fine-tuning, because removing it introduced a pre-train/fine-tune mismatch that degraded downstream performance. Additionally, this model was pre-trained without the NSP task. This is directly comparable to OpenAI GPT, but using our larger training dataset, our input representation, and our fine-tuning scheme.

We first examine the impact brought by the NSP task. In Table 5, we show that removing NSP hurts performance significantly on QNLI, MNLI, and SQuAD 1.1. Next, we evaluate the impact of training bidirectional representations by comparing “No NSP” to “LTR & No NSP”. The LTR model performs worse than the MLM model on all tasks, with large drops on MRPC and SQuAD.

For SQuAD it is intuitively clear that a LTR model will perform poorly at token predictions, since the token-level hidden states have no right-side context. In order to make a good faith attempt at strengthening the LTR system, we added a randomly initialized BiLSTM on top. This does significantly improve results on SQuAD, but the

results are still far worse than those of the pre-trained bidirectional models. The BiLSTM hurts performance on the GLUE tasks.

We recognize that it would also be possible to train separate LTR and RTL models and represent each token as the concatenation of the two models, as ELMo does. However: (a) this is twice as expensive as a single bidirectional model; (b) this is non-intuitive for tasks like QA, since the RTL model would not be able to condition the answer on the question; (c) this it is strictly less powerful than a deep bidirectional model, since it can use both left and right context at every layer.

5.2 Effect of Model Size

In this section, we explore the effect of model size on fine-tuning task accuracy. We trained a number of BERT models with a differing number of layers, hidden units, and attention heads, while otherwise using the same hyperparameters and training procedure as described previously.

Results on selected GLUE tasks are shown in Table 6. In this table, we report the average Dev Set accuracy from 5 random restarts of fine-tuning. We can see that larger models lead to a strict accuracy improvement across all four datasets, even for MRPC which only has 3,600 labeled training examples, and is substantially different from the pre-training tasks. It is also perhaps surprising that we are able to achieve such significant improvements on top of models which are already quite large relative to the existing literature. For example, the largest Transformer explored in Vaswani et al. (2017) is (L=6, H=1024, A=16) with 100M parameters for the encoder, and the largest Transformer we have found in the literature is (L=64, H=512, A=2) with 235M parameters (Al-Rfou et al., 2018). By contrast, BERT_{BASE} contains 110M parameters and BERT_{LARGE} contains 340M parameters.

It has long been known that increasing the model size will lead to continual improvements on large-scale tasks such as machine translation and language modeling, which is demonstrated by the LM perplexity of held-out training data shown in Table 6. However, we believe that this is the first work to demonstrate convincingly that scaling to extreme model sizes also leads to large improvements on very small scale tasks, provided that the model has been sufficiently pre-trained. Peters et al. (2018b) presented

任务	开发集				
	MNLI-m (准确率)	QNLI (准确率)	MRPC (准确率)	SST-2 (准确率)	SQuAD (F1)
BERT _{BASE}	84.4	88.4	86.7	92.7	88.5
无 NSP	83.9	84.9	86.5	92.6	87.9
LTR & 无 NSP	82.1	84.3	77.5	92.1	77.8
+ BiLSTM	82.1	84.1	75.7	91.6	84.9

表 5: 基于 BERT_{BASE} 架构的预训练任务消融实验。“无 NSP” 的训练去除了下一句预测任务。“LTR & 无 NSP” 按左至右语言模型进行训练, 且不包含下一句预测任务, 类似于 OpenAI GPT。“+ BiLSTM” 则是在微调期间, 于 “LTR + 无 NSP” 模型之上叠加了一个随机初始化的 BiLSTM。

相关的消融研究可参见附录C。

5.1 预训练任务的影响

我们通过评估两种预训练目标来证明BERT深度双向性的重要性, 这两种目标采用了与 BERT_{BASE}完全相同的预训练数据、微调方案和超参数:

No NSP: 一个采用“掩码语言模型”(MLM) 进行训练的双向模型, 但不包含“下一句预测”(NSP) 任务。

LTR & No NSP: 一种仅依赖左侧上下文的模型, 使用标准的从左到右 (LTR) 语言模型而非 MLM进行训练。该左侧约束同样应用于微调阶段, 因为移除它会导致预训练与微调之间的不匹配, 进而降低下游任务性能。此外, 该模型在预训练阶段也未使用NSP任务。这与OpenAI GPT具有直接可比性, 但基于我们更大的训练数据集、我们的输入表示方式以及我们的微调方案。

我们首先考察NSP任务所带来的影响。在表 5中, 我们表明移除NSP会对QNLI、MNLI和SQuAD 1.1的性能造成显著损害。接下来, 我们通过对比“无NSP”与“LTR & No NSP”来评估训练双向表示的影响。LTR模型在所有任务上的表现均逊于MLM模型, 在MRPC和SQuAD上的性能下降尤为明显。

对于SQuAD任务而言, 显然LTR模型在词元预测上的表现较差, 因为词元级的隐藏状态缺乏右侧上下文信息。为切实尝试增强LTR系统, 我们在其上方添加了一个随机初始化的BiLSTM。这确实在SQuAD上显著提升了结果, 但

结果仍远逊于预训练双向模型。BiLSTM的引入反而降低了GLUE任务的性能。

我们认识到, 也可以像ELMo那样分别训练从左到右 (LTR) 和从右到左 (RTL) 的模型, 并将每个词元表示为这两个模型结果的拼接。然而: (a) 其成本是单个双向模型的两倍; (b) 这对于问答等任务来说不够直观, 因为RTL模型无法以问题为条件来生成答案; (c) 它的表达能力严格弱于深度双向模型, 因为它仅能在每一层同时利用左侧和右侧的上下文。

5.2 模型规模的影响

在本节中, 我们探讨模型规模对微调任务准确率的影响。我们训练了多个BERT变体, 它们在层数、隐藏单元数和注意力头数上各不相同, 其余配置均沿用前文所述的相同超参数与训练流程。

Results on selected GLUE tasks are shown in Table 6. In this table, we report the average Dev Set accuracy from 5 random restarts of fine-tuning. We can see that larger models lead to a strict accuracy improvement across all four datasets, even for MRPC which only has 3,600 labeled training examples, and is substantially different from the pre-training tasks. It is also perhaps surprising that we are able to achieve such significant improvements on top of models which are already quite large relative to the existing literature. For example, the largest Transformer explored in Vaswani et al. (2017) is (L=6, H=1024, A=16) with 100M parameters for the encoder, and the largest Transformer we have found in the literature is (L=64, H=512, A=2) with 235M parameters (Al-Rfou et al., 2018). By contrast, BERT_{BASE} contains 110M parameters and BERT_{LARGE} contains 340M parameters.

长期以来人们就已知晓, 增加模型规模将能持续提升机器翻译和语言建模等大规模任务的性能, 这已通过表中所示未见训练数据的语言模型困惑度得到证实。6。然而, 我们认为这是首次令人信服地证明, 只要模型经过充分预训练, 扩展至极大规模同样能为极小规模任务带来显著提升。Peters等人(2018b)曾展示过

mixed results on the downstream task impact of increasing the pre-trained bi-LM size from two to four layers and Melamud et al. (2016) mentioned in passing that increasing hidden dimension size from 200 to 600 helped, but increasing further to 1,000 did not bring further improvements. Both of these prior works used a feature-based approach — we hypothesize that when the model is fine-tuned directly on the downstream tasks and uses only a very small number of randomly initialized additional parameters, the task-specific models can benefit from the larger, more expressive pre-trained representations even when downstream task data is very small.

5.3 Feature-based Approach with BERT

All of the BERT results presented so far have used the fine-tuning approach, where a simple classification layer is added to the pre-trained model, and all parameters are jointly fine-tuned on a downstream task. However, the feature-based approach, where fixed features are extracted from the pre-trained model, has certain advantages. First, not all tasks can be easily represented by a Transformer encoder architecture, and therefore require a task-specific model architecture to be added. Second, there are major computational benefits to pre-compute an expensive representation of the training data once and then run many experiments with cheaper models on top of this representation.

In this section, we compare the two approaches by applying BERT to the CoNLL-2003 Named Entity Recognition (NER) task (Tjong Kim Sang and De Meulder, 2003). In the input to BERT, we use a case-preserving WordPiece model, and we include the maximal document context provided by the data. Following standard practice, we formulate this as a tagging task but do not use a CRF

Hyperparams				Dev Set Accuracy		
#L	#H	#A	LM (ppl)	MNLI-m	MRPC	SST-2
3	768	12	5.84	77.9	79.8	88.4
6	768	3	5.24	80.6	82.2	90.7
6	768	12	4.68	81.9	84.8	91.3
12	768	12	3.99	84.4	86.7	92.9
12	1024	16	3.54	85.7	86.9	93.3
24	1024	16	3.23	86.6	87.8	93.7

Table 6: Ablation over BERT model size. #L = the number of layers; #H = hidden size; #A = number of attention heads. “LM (ppl)” is the masked LM perplexity of held-out training data.

System	Dev F1	Test F1
ELMo (Peters et al., 2018a)	95.7	92.2
CVT (Clark et al., 2018)	-	92.6
CSE (Akbik et al., 2018)	-	93.1
Fine-tuning approach		
BERT _{LARGE}	96.6	92.8
BERT _{BASE}	96.4	92.4
Feature-based approach (BERT _{BASE})		
Embeddings	91.0	-
Second-to-Last Hidden	95.6	-
Last Hidden	94.9	-
Weighted Sum Last Four Hidden	95.9	-
Concat Last Four Hidden	96.1	-
Weighted Sum All 12 Layers	95.5	-

Table 7: CoNLL-2003 Named Entity Recognition results. Hyperparameters were selected using the Dev set. The reported Dev and Test scores are averaged over 5 random restarts using those hyperparameters.

layer in the output. We use the representation of the first sub-token as the input to the token-level classifier over the NER label set.

To ablate the fine-tuning approach, we apply the feature-based approach by extracting the activations from one or more layers *without* fine-tuning any parameters of BERT. These contextual embeddings are used as input to a randomly initialized two-layer 768-dimensional BiLSTM before the classification layer.

Results are presented in Table 7. BERT_{LARGE} performs competitively with state-of-the-art methods. The best performing method concatenates the token representations from the top four hidden layers of the pre-trained Transformer, which is only 0.3 F1 behind fine-tuning the entire model. This demonstrates that BERT is effective for both fine-tuning and feature-based approaches.

6 Conclusion

Recent empirical improvements due to transfer learning with language models have demonstrated that rich, unsupervised pre-training is an integral part of many language understanding systems. In particular, these results enable even low-resource tasks to benefit from deep unidirectional architectures. Our major contribution is further generalizing these findings to deep *bidirectional* architectures, allowing the same pre-trained model to successfully tackle a broad set of NLP tasks.

关于预训练双向语言模型 (bi-LM) 规模从两层增加到四层对下游任务影响的结论不一, 以及 Melamud 等人(2016)曾顺带提及, 将隐藏维度从200提升至600可带来改善, 但继续增至1000则未产生进一步收益。上述两项既往研究均采用基于特征的方法——我们推测, 当模型直接在下游任务上进行微调, 且仅引入极少量随机初始化的附加参数时, 即便下游任务的数据量极为匮乏, 特定任务的模型依然能够得益于规模更大、表达能力更强的预训练表征。

5.3 基于特征的 BERT 方法

截至目前介绍的所有 BERT 结果均采用微调方法, 即在预训练模型的基础上添加一个简单的分类层, 并将所有参数在下游任务上进行联合微调。然而, 基于特征的方法 (即从预训练模型中直接提取固定特征) 具有一定的优势。首先, 并非所有任务都能简单地由 Transformer 编码器架构来表示, 因此需要额外添加针对特定任务的模型架构。其次, 一次性预先计算训练数据中计算代价高昂的表征具有显著的计算优势, 之后可在此基础上使用开销较低的模型运行大量实验。

在本节中, 我们通过将 BERT 应用于 CoNLL-2003 命名实体识别 (NER) 任务来比较这两种方法 (Tjong Kim Sang 和 De Meulder, 2003)。在 BERT 的输入中, 我们采用保留大小写的 WordPiece 模型, 并包含数据提供的完整文档上下文。遵循标准做法, 我们将其构建为序列标注任务, 但不使用 CRF

超参数				验证集准确率		
#L	隐藏层数 (#H)	注意力头数 (#A)	语言模型 (LM, ppl)	MNLI-m	MRPC	SST-2
3	768	12	5.84	77.9	79.8	88.4
6	768	3	5.24	80.6	82.2	90.7
6	768	12	4.68	81.9	84.8	91.3
12	768	12	3.99	84.4	86.7	92.9
12	1024	16	3.54	85.7	86.9	93.3
24	1024	16	3.23	86.6	87.8	93.7

表6: BERT模型规模的消融实验。#L = 表示层数; #H = 表示隐藏层大小; #A = 表示注意力头数量。 “LM (ppl)”表示掩码语言模型在预训练数据上的困惑度。

系统	开发集 F1	测试集 F1
ELMo (Peters 等, 2018a)	95.7	92.2
CVT (Clark 等, 2018)	-	92.6
CSE (Akbik 等, 2018)	-	93.1
微调方法		
BERT _{LARGE}	96.6	92.8
BERT _{BASE}	96.4	92.4
基于特征的方法 (BERT _{BASE})		
嵌入	91.0	-
倒数第二层隐藏状态	95.6	-
最后一层隐藏状态	94.9	-
最后四层隐藏状态的加权和	95.9	-
拼接最后四个隐藏层	96.1	-
对全部12层进行加权求和	95.5	-

表7: CoNLL-2003命名实体识别结果。超参数是通过开发集 (Dev set) 选择的。报告的Dev和Test得分是使用这些超参数在5次随机重启中取平均值的结果。

层输出中。我们使用第一个子词元的表征作为针对 NER 标签集的 Token 级分类器的输入。

为了进行消融实验, 我们采用基于特征的方法, 从一到多个层中提取激活状态, 且 *without* 对BERT的任何参数进行微调。这些上下文嵌入被用作分类层前随机初始化的双层768维BiLSTM的输入。

结果如表7所示。BERT_{LARGE} 展现出具有竞争力的表现, 可与最先进的方法相媲美。表现最佳的方法拼接了预训练Transformer顶部四个隐藏层的词元表示, 其F1分数仅比微调整个模型低0.3。这表明BERT既适用于微调方法, 也适用于基于特征的方法。

6 结论

近年来, 基于语言模型的迁移学习所带来的实证改进表明, 丰富且无监督的预训练已成为许多语言理解系统不可或缺的一部分。特别是, 这些结果表明, 即使是低资源任务也能从深度单向架构中受益。我们的主要贡献在于将这些发现进一步推广到深度*bidirectional* 架构, 使得同一预训练模型能够成功应对广泛的NLP任务。

References

Alan Akbik, Duncan Blythe, and Roland Vollgraf. 2018. Contextual string embeddings for sequence labeling. In *Proceedings of the 27th International Conference on Computational Linguistics*, pages 1638–1649.

Rami Al-Rfou, Dokook Choe, Noah Constant, Mandy Guo, and Llion Jones. 2018. Character-level language modeling with deeper self-attention. *arXiv preprint arXiv:1808.04444*.

Rie Kubota Ando and Tong Zhang. 2005. A framework for learning predictive structures from multiple tasks and unlabeled data. *Journal of Machine Learning Research*, 6(Nov):1817–1853.

Luisa Bentivogli, Bernardo Magnini, Ido Dagan, Hoa Trang Dang, and Danilo Giampiccolo. 2009. The fifth PASCAL recognizing textual entailment challenge. In *TAC*. NIST.

John Blitzer, Ryan McDonald, and Fernando Pereira. 2006. Domain adaptation with structural correspondence learning. In *Proceedings of the 2006 conference on empirical methods in natural language processing*, pages 120–128. Association for Computational Linguistics.

Samuel R. Bowman, Gabor Angeli, Christopher Potts, and Christopher D. Manning. 2015. A large annotated corpus for learning natural language inference. In *EMNLP*. Association for Computational Linguistics.

Peter F Brown, Peter V Desouza, Robert L Mercer, Vincent J Della Pietra, and Jenifer C Lai. 1992. Class-based n-gram models of natural language. *Computational linguistics*, 18(4):467–479.

Daniel Cer, Mona Diab, Eneko Agirre, Inigo Lopez-Gazpio, and Lucia Specia. 2017. **Semeval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation**. In *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*, pages 1–14, Vancouver, Canada. Association for Computational Linguistics.

Ciprian Chelba, Tomas Mikolov, Mike Schuster, Qi Ge, Thorsten Brants, Phillipp Koehn, and Tony Robinson. 2013. One billion word benchmark for measuring progress in statistical language modeling. *arXiv preprint arXiv:1312.3005*.

Z. Chen, H. Zhang, X. Zhang, and L. Zhao. 2018. **Quora question pairs**.

Christopher Clark and Matt Gardner. 2018. Simple and effective multi-paragraph reading comprehension. In *ACL*.

Kevin Clark, Minh-Thang Luong, Christopher D Manning, and Quoc Le. 2018. Semi-supervised sequence modeling with cross-view training. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 1914–1925.

Ronan Collobert and Jason Weston. 2008. A unified architecture for natural language processing: Deep neural networks with multitask learning. In *Proceedings of the 25th international conference on Machine learning*, pages 160–167. ACM.

Alexis Conneau, Douwe Kiela, Holger Schwenk, Loïc Barrault, and Antoine Bordes. 2017. **Supervised learning of universal sentence representations from natural language inference data**. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 670–680, Copenhagen, Denmark. Association for Computational Linguistics.

Andrew M Dai and Quoc V Le. 2015. Semi-supervised sequence learning. In *Advances in neural information processing systems*, pages 3079–3087.

J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. 2009. ImageNet: A Large-Scale Hierarchical Image Database. In *CVPR09*.

William B Dolan and Chris Brockett. 2005. Automatically constructing a corpus of sentential paraphrases. In *Proceedings of the Third International Workshop on Paraphrasing (IWP2005)*.

William Fedus, Ian Goodfellow, and Andrew M Dai. 2018. Maskgan: Better text generation via filling in the... *arXiv preprint arXiv:1801.07736*.

Dan Hendrycks and Kevin Gimpel. 2016. **Bridging nonlinearities and stochastic regularizers with gaussian error linear units**. *CoRR*, abs/1606.08415.

Felix Hill, Kyunghyun Cho, and Anna Korhonen. 2016. Learning distributed representations of sentences from unlabelled data. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics.

Jeremy Howard and Sebastian Ruder. 2018. **Universal language model fine-tuning for text classification**. In *ACL*. Association for Computational Linguistics.

Minghao Hu, Yuxing Peng, Zhen Huang, Xipeng Qiu, Furu Wei, and Ming Zhou. 2018. Reinforced mnemonic reader for machine reading comprehension. In *IJCAI*.

Yacine Jernite, Samuel R. Bowman, and David Son-tag. 2017. **Discourse-based objectives for fast unsupervised sentence representation learning**. *CoRR*, abs/1705.00557.

参考文献

Alan Akbik, Duncan Blythe, and Roland Vollgraf。2018年。用于序列标注的上下文字符串嵌入。收录于*Proceedings of the27th InternationalConferenceonComputational Linguistics*, 第1638–1649页。

Rami Al-Rfou, Dokook Choe, Noah Constant, MandyGuo, and Llion Jones. 2018. 具有更深自注意力机制的字级别语言建模。arXiv预印本 arXiv:1808.04444。

Rie Kubota Ando and Tong Zhang. 2005. 从多任务和无标签数据中学习预测结构的框架。《机器学习研究杂志》, 6(Nov):1817–1853。

Luisa Bentivogli, Bernardo Magnini, Ido Dagan,Hoa Trang Dang, and Danilo Giampiccolo. 2009.第五届PASCAL 文本蕴含识别挑战赛。见*TAC*。NIST。

John Blitzer, Ryan McDonald, and Fernando Pereira.2006. 基于结构对应学习的领域自适应。收录于《*2006年自然语言处理实证方法会议论文集*》, 第120–128页。计算语言学协会。

Samuel R. Bowman, Gabor Angeli, Christopher Potts,以及Christopher D. Manning. 2015. 用于学习自然语言推理的大型标注语料库.收录于 *EMNLP*. 计算语言学协会。Peter F Brown, Peter V Desouza, Robert L Mercer,Vincent J Della Pietra, 以及Jenifer C Lai. 1992.基于类别的自然语言n元语法模型。《计算语言学》, 18(4):467–479.Daniel Cer, Mona Diab, Eneko Agirre, Inigo Lopez-Gazpio, 以及Lucia Specia. 2017. SemEval-2017任务1: 多语言与跨语言语义文本相似度专项评估. 收录于 《第11届语义评估国际研讨会论文集（SemEval-2017）》, 第1–14页, 范库-\n弗, 加拿大. 计算语言学协会.Ciprian Chelba, Tomas Mikolov, Mike Schuster, Qi Ge,Thorsten Brants, Phillipp Koehn, 以及 Tony Robin-son. 2013. 用于衡量统计语言建模进展的十亿词基准测试. arXiv预印本 arXiv:1312.3005.Z. Chen, H. Zhang, X. Zhang, 以及L. Zhao. 2018.Quora 问题对. Christopher Clark和Matt Gardner. 2018. 简单高效的多段落阅读理解. 收录于*ACL*。

Kevin Clark, Minh-Thang Luong, Christopher D Manning 以及 Quoc Le. 2018年。基于交叉视图训练的半监督序列建模。在 *2018年自然语言处理实证方法大会论文集*, 第1914–1925页。

Ronan Collobert 与 Jason Weston. 2008年。面向自然语言处理的统一架构：基于多任务学习的深度神经网络。收录于第25届国际机器学习会议论文集, 第160–167页。ACM。

Alexis Conneau、Douwe Kiela、Holger Schwenk、Lo’icBarrault 与 Antoine Bordes。2017年。**基于自然语言推理数据的通用句子表示监督学习**。收录于*2017年自然语言处理实证方法会议论文集*, 第670–680页, 丹麦哥本哈根。计算语言学协会。

Andrew M Dai 与 Quoc V Le. 2015年。半监督序列学习。收录于神经信息处理系统进展, 第3079–3087页。

J. Deng、W. Dong、R. Socher、L.-J. Li、K. Li 与 L. Fei-Fei。2009年。ImageNet：大规模层次化图像数据库。收录于*CVPR09*会议论文集。

William B Dolan 与 Chris Brockett. 2005年。自动构建句子释义语料库。发表于 第三届国际重述研讨会论文集(IWP2005)。William Fedus, Ian Goodfellow 与 Andrew M Dai。2018年。MaskGAN：通过填充实现更优文本生成。*arXiv*预印本 *arXiv:1801.07736*。Dan Hendrycks 与 Kevin Gimpel。2016年。利用高斯误差线性单元桥接非线性项与随机正则化器。*CoRR*, abs/1606.08415。Felix Hill, Kyunghyun Cho 与 Anna Korhonen。2016年。从非标注数据中学习句子的分布式表示。收录于2016年北美计算语言学协会年会论文集：人类语言技术。计算语言学协会出版。Jeremy Howard 与 Sebastian Ruder。2018年。面向文本分类的通用语言模型微调。收录于ACL。计算语言学协会出版。Minghao Hu, Yuxing Peng, Zhen Huang, Xipeng Qiu,Furu Wei 与 Ming Zhou。2018年。用于机器阅读理解的强化记忆读取模型。收录于*IJCAI*。Yacine Jernite, Samuel R. Bowman 与 David Son-tag。2017年。**用于快速无监督句子表示学习的话语级目标函数**。*CoRR*, abs/1705.00557。

Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. 2017. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In *ACL*.

Ryan Kiros, Yukun Zhu, Ruslan R Salakhutdinov, Richard Zemel, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. 2015. Skip-thought vectors. In *Advances in neural information processing systems*, pages 3294–3302.

Quoc Le and Tomas Mikolov. 2014. Distributed representations of sentences and documents. In *International Conference on Machine Learning*, pages 1188–1196.

Hector J Levesque, Ernest Davis, and Leora Morgenstern. 2011. The winograd schema challenge. In *Aaai spring symposium: Logical formalizations of commonsense reasoning*, volume 46, page 47.

Lajanugen Logeswaran and Honglak Lee. 2018. [An efficient framework for learning sentence representations](#). In *International Conference on Learning Representations*.

Bryan McCann, James Bradbury, Caiming Xiong, and Richard Socher. 2017. Learned in translation: Contextualized word vectors. In *NIPS*.

Oren Melamud, Jacob Goldberger, and Ido Dagan. 2016. context2vec: Learning generic context embedding with bidirectional LSTM. In *CoNLL*.

Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems 26*, pages 3111–3119. Curran Associates, Inc.

Andriy Mnih and Geoffrey E Hinton. 2009. [A scalable hierarchical distributed language model](#). In D. Koller, D. Schuurmans, Y. Bengio, and L. Bottou, editors, *Advances in Neural Information Processing Systems 21*, pages 1081–1088. Curran Associates, Inc.

Ankur P Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. 2016. A decomposable attention model for natural language inference. In *EMNLP*.

Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. [Glove: Global vectors for word representation](#). In *Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543.

Matthew Peters, Waleed Ammar, Chandra Bhagavatula, and Russell Power. 2017. Semi-supervised sequence tagging with bidirectional language models. In *ACL*.

Matthew Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. 2018a. Deep contextualized word representations. In *NAACL*.

Matthew Peters, Mark Neumann, Luke Zettlemoyer, and Wen-tau Yih. 2018b. Dissecting contextual word embeddings: Architecture and representation. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 1499–1509.

Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving language understanding with unsupervised learning. Technical report, OpenAI.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. Squad: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392.

Minjoon Seo, Aniruddha Kembhavi, Ali Farhadi, and Hannaneh Hajishirzi. 2017. Bidirectional attention flow for machine comprehension. In *ICLR*.

Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Ng, and Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment tree-bank. In *Proceedings of the 2013 conference on empirical methods in natural language processing*, pages 1631–1642.

Fu Sun, Linyang Li, Xipeng Qiu, and Yang Liu. 2018. U-net: Machine reading comprehension with unanswerable questions. *arXiv preprint arXiv:1810.06638*.

Wilson L Taylor. 1953. Cloze procedure: A new tool for measuring readability. *Journalism Bulletin*, 30(4):415–433.

Erik F Tjong Kim Sang and Fien De Meulder. 2003. Introduction to the conll-2003 shared task: Language-independent named entity recognition. In *CoNLL*.

Joseph Turian, Lev Ratinov, and Yoshua Bengio. 2010. Word representations: A simple and general method for semi-supervised learning. In *Proceedings of the 48th Annual Meeting of the Association for Computational Linguistics, ACL ’10*, pages 384–394.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, pages 6000–6010.

Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. 2008. Extracting and composing robust features with denoising autoencoders. In *Proceedings of the 25th international conference on Machine learning*, pages 1096–1103. ACM.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. 2018a. Glue: A multi-task benchmark and analysis platform

Mandar Joshi, Eunsol Choi, Daniel S Weld 与 LukeZettlemoyer。2017年。TriviaQA：一个用于阅读理解的大规模远监督挑战数据集。收录于*ACL*。

Ryan Kiros, Yukun Zhu, Ruslan R Salakhutdinov,Richard Zemel, Raquel Urtasun, Antonio Torralba,以及 Sanja Fidler。2015。Skip-thought 向量。收录于神经信息处理系统进展,第 3294–3302 页。

Quoc Le 和 Tomas Mikolov。2014。句子与文档的分布式表示。发表于 机器学习国际会议, 第 1188–1196 页。

Hector J Levesque, Ernest Davis, 和 Leora Morgenstern。2011。Winograd 范式挑战。发表于AAAI春季研讨会：常识推理的逻辑形式化, 第 46 卷, 第 47 页。

Lajanugen Logeswaran 和 Honglak Lee。2018。一种用于学习句子表示的高效框架。发表于 学习表示国际会议。

Bryan McCann, James Bradbury, Caiming Xiong, 和 Richard Socher。2017。翻译中的学习：上下文词向量。收录于*NIPS*。

Oren Melamud、Jacob Goldberger 与 Ido Dagan。2016年。context2vec：利用双向LSTM学习通用上下文嵌入。发表于*CoNLL*。Tomas Mikolov、Ilya Sutskever、Kai Chen、Greg S Corrado 与 Jeff Dean。2013年。词汇与短语的分布式表征及其组合性。发表于《神经信息处理系统进展》26, 第 3111–3119 页。Curran Associates,Inc。Andriy Mnih 与 Geoffrey E Hinton。2009年。[可扩展的分层分布式语言模型](#)。发表于由 D. Koller、D. Schuurmans、Y. Bengio 与 L. Bot-tou 主编,《神经信息处理系统进展》21, 第 1081–1088 页。Curran Associates, Inc。Ankur P Parikh、Oscar Täckström、Dipanjan Das 与Jakob Uszkoreit。2016年。面向自然语言推理的可分解注意力模型。发表于*EMNLP*。Jeffrey Pennington、Richard Socher 与 Christopher D. Manning。2014年。[GloVe：用于词表示的全局向量](#)。发表于 自然语言处理实证方法 (*EMNLP*) , 第 1532–1543 页。Matthew Peters、Waleed Ammar、Chandra Bhagavat-ula 与 Russell Power。2017年。基于双向语言模型的半监督序列标注。发表于 *ACL*。

Matthew Peters, Mark Neumann, Mohit Iyyer, MattGardner, Christopher Clark, Kenton Lee, and LukeZettlemoyer。2018a。深度语境化词表示。发表于*NAACL*。

Matthew Peters, Mark Neumann, Luke Zettlemoyer,和 Wen-tau Yih。2018b。解构语境化词嵌入：架构与表示。发表于 2018年自然语言处理实证方法会议论文集, 第1499–1509页。

Alec Radford, Karthik Narasimhan, Tim Salimans, 和Ilya Sutskever。2018。利用无监督学习改进语言理解。OpenAI 技术报告。

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, 和Percy Liang。2016。SQuAD：面向文本机器理解的 100,000+ 个问答。收录于 2016年自然语言处理实证方法会议论文集, 第 2383–2392 页。

Minjoon Seo, Aniruddha Kembhavi, Ali Farhadi, 和 Hannaneh Hajishirzi。2017。用于机器阅读理解的双向注意力流。收录于 *ICLR*。

Richard Socher, Alex Perelygin, Jean Wu, JasonChuang, Christopher D Manning, Andrew Ng, 和Christopher Potts。2013。基于情感树库实现语义组合性的递归深度模型。收录于 2013年自然语言处理实证方法会议论文集, 第 1631–1642 页。

Fu Sun, Linyang Li, Xipeng Qiu, and Yang Liu.2018. U-net：面向不可回答问题的机器阅读理解。arXiv预印本arXiv:1810.06638.Wilson L Taylor. 1953. 完形填空法：一种测量可读性的新工具。《新闻公报》,30(4):415–433.Erik F Tjong Kim Sang and Fien De Meulder.2003. CoNLL-2003共享任务简介：语言无关的命名实体识别。收录于CoNLL.Joseph Turian, Lev Ratinov, and Yoshua Bengio. 2010.词表示：一种用于半监督学习的简单通用方法。在《第48届计算语言学协会年会论文集》, ACL ’ 10, 第384–394页。Ashish Vaswani, Noam Shazeer, Niki Parmar, JakobUszkoreit, Llion Jones, Aidan N Gomez, LukaszKaiser, and Illia Polosukhin. 2017. 注意力即是一切。在《神经信息处理系统进展》,第6000–6010页。Pascal Vincent, Hugo Larochelle, Yoshua Bengio, andPierre-Antoine Manzagol. 2008. 利用去噪自编码-器提取与组合鲁棒特征。在《第25届国际机器学习会议论文集》, 第1096–1103页。ACM。Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. 2018a.GLUE：多任务基准与分析平台

for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 353–355.

Wei Wang, Ming Yan, and Chen Wu. 2018b. Multi-granularity hierarchical attention fusion networks for reading comprehension and question answering. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics.

Alex Warstadt, Amanpreet Singh, and Samuel R Bowman. 2018. Neural network acceptability judgments. *arXiv preprint arXiv:1805.12471*.

Adina Williams, Nikita Nangia, and Samuel R Bowman. 2018. A broad-coverage challenge corpus for sentence understanding through inference. In *NAACL*.

Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. 2016. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*.

Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. 2014. How transferable are features in deep neural networks? In *Advances in neural information processing systems*, pages 3320–3328.

Adams Wei Yu, David Dohan, Minh-Thang Luong, Rui Zhao, Kai Chen, Mohammad Norouzi, and Quoc V Le. 2018. QANet: Combining local convolution with global self-attention for reading comprehension. In *ICLR*.

Rowan Zellers, Yonatan Bisk, Roy Schwartz, and Yejin Choi. 2018. Swag: A large-scale adversarial dataset for grounded commonsense inference. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Yukun Zhu, Ryan Kiros, Rich Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. 2015. Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. In *Proceedings of the IEEE international conference on computer vision*, pages 19–27.

Appendix for “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”

We organize the appendix into three sections:

- Additional implementation details for BERT are presented in Appendix A;

- Additional details for our experiments are presented in Appendix B; and

- Additional ablation studies are presented in Appendix C.

We present additional ablation studies for BERT including:

- Effect of Number of Training Steps; and
- Ablation for Different Masking Procedures.

A Additional Details for BERT

A.1 Illustration of the Pre-training Tasks

We provide examples of the pre-training tasks in the following.

Masked LM and the Masking Procedure Assuming the unlabeled sentence is `my dog is hairy`, and during the random masking procedure we chose the 4-th token (which corresponding to `hairy`), our masking procedure can be further illustrated by

- 80% of the time: Replace the word with the [MASK] token, e.g., `my dog is hairy` → `my dog is [MASK]`
- 10% of the time: Replace the word with a random word, e.g., `my dog is hairy` → `my dog is apple`
- 10% of the time: Keep the word unchanged, e.g., `my dog is hairy` → `my dog is hairy`. The purpose of this is to bias the representation towards the actual observed word.

The advantage of this procedure is that the Transformer encoder does not know which words it will be asked to predict or which have been replaced by random words, so it is forced to keep a distributional contextual representation of every input token. Additionally, because random replacement only occurs for 1.5% of all tokens (i.e., 10% of 15%), this does not seem to harm the model’s language understanding capability. In Section C.2, we evaluate the impact this procedure.

Compared to standard language model training, the masked LM only make predictions on 15% of tokens in each batch, which suggests that more pre-training steps may be required for the model

用于自然语言理解。在《2018年EMNLP BlackboxNLP研讨会论文集：面向自然语言处理的神经网络分析与解释》，第353–355页。

Wei Wang, Ming Yan, and Chen Wu. 2018b. 面向阅读理解与问答的多粒度层次注意力融合网络。收录于第56届计算语言学协会年会论文集（第1卷：长篇论文）。计算语言学协会。

Alex Warstadt, Amanpreet Singh, and Samuel R Bowman. 2018. 神经网络的接受度评判。arXiv预印本 arXiv:1805.12471。

Adina Williams, Nikita Nangia, and Samuel R Bowman. 2018. 基于推理的句子理解广覆盖挑战语料库。收录于NAACL。

Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. 2016. Google的神经机器翻译系统：弥合人类与机器翻译之间的差距。arXiv预印本 arXiv:1609.08144。

Jason Yosinski, Jeff Clune, Yoshua Bengio 与 Hod Lipson. 2014. 深度神经网络中特征的迁移性如何？在神经信息处理系统进展中，第3320–3328页。

Adams Wei Yu, David Dohan, Minh-Thang Luong, Rui Zhao, Kai Chen, Mohammad Norouzi 与 Quoc V Le. 2018. QANet：结合局部卷积与全局自注意力进行阅读理解。在ICLR中。

Rowan Zellers, Yonatan Bisk, Roy Schwartz 与 Yejin Choi. 2018. SWAG：面向基础常识推理的大规模对抗数据集。收录于2018年自然语言处理实证方法会议论文集（EMNLP）。

Yukun Zhu, Ryan Kiros, Rich Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba 与 Sanja Fidler. 2015. 对齐图书与电影：通过观看电影和阅读图书实现故事化的视觉解释。收录于IEEE国际计算机视觉会议论文集，第19–27页。

“BERT：用于语言理解的深度双向Transformer预训练”附录

我们将附录分为三个部分：

- 关于 BERT 的实现细节详见附录A；

- 实验的相关细节详见附录 B；以及

- 额外的消融实验展示于附录 C。

我们针对BERT提供了更多的消融实验，包括：

- 训练步数的影响；以及– 不同掩码方法的消融实验。

A BERT的更多详细信息

A.1 预训练任务的图解

以下提供预训练任务的示例。

掩码语言模型与掩码过程 假设无标签句子为 `my dogishairy`，在随机掩码过程中我们选择了第4个词元（对应于 `hairy`），我们的掩码过程可进一步说明如下：

- 80%的情况下：将该词替换为[MASK]词元，例如 `my dog is hairy` → `my dog is [MASK]`

- 10%的情况下：将该词替换为一个随机词，例如 `my dog is hairy` → `my dog is apple`

- 10%的情况下：保持该词不变，例如，`my dog is hairy` → `my dog is hairy`。此步骤的目的是使表示偏向于实际观测到的词语。

该过程的优势在于，Transformer编码器无法获知将被要求预测哪些词，也无法知晓哪些词已被随机词替换，因此它被迫为每一个输入标记保留分布式的上下文表示。此外，由于随机替换仅涉及全部标记的1.5%（即15%的10%），这似乎不会损害模型的语言理解能力。在C.2节中，我们将评估该过程的影响。

与标准语言模型训练相比，掩码语言模型仅对每个批次中 15% 的标记进行预测，这表明该模型可能需要更多的预训练步骤。

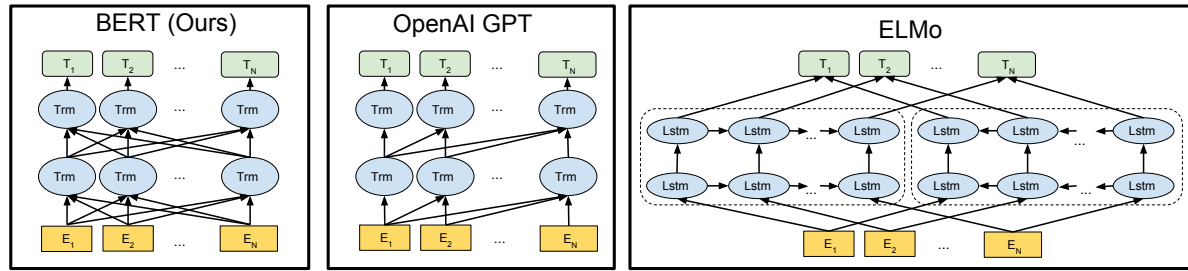


Figure 3: Differences in pre-training model architectures. BERT uses a bidirectional Transformer. OpenAI GPT uses a left-to-right Transformer. ELMo uses the concatenation of independently trained left-to-right and right-to-left LSTMs to generate features for downstream tasks. Among the three, only BERT representations are jointly conditioned on both left and right context in all layers. In addition to the architecture differences, BERT and OpenAI GPT are fine-tuning approaches, while ELMo is a feature-based approach.

to converge. In Section C.1 we demonstrate that MLM does converge marginally slower than a left-to-right model (which predicts every token), but the empirical improvements of the MLM model far outweigh the increased training cost.

Next Sentence Prediction The next sentence prediction task can be illustrated in the following examples.

Input = [CLS] the man went to [MASK] store [SEP]
he bought a gallon [MASK] milk [SEP]
Label = IsNext

Input = [CLS] the man [MASK] to the store [SEP]
penguin [MASK] are flight ##less birds [SEP]
Label = NotNext

A.2 Pre-training Procedure

To generate each training input sequence, we sample two spans of text from the corpus, which we refer to as “sentences” even though they are typically much longer than single sentences (but can be shorter also). The first sentence receives the A embedding and the second receives the B embedding. 50% of the time B is the actual next sentence that follows A and 50% of the time it is a random sentence, which is done for the “next sentence prediction” task. They are sampled such that the combined length is ≤ 512 tokens. The LM masking is applied after WordPiece tokenization with a uniform masking rate of 15%, and no special consideration given to partial word pieces.

We train with batch size of 256 sequences (256 sequences * 512 tokens = 128,000 tokens/batch) for 1,000,000 steps, which is approximately 40

epochs over the 3.3 billion word corpus. We use Adam with learning rate of $1e-4$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, L2 weight decay of 0.01, learning rate warmup over the first 10,000 steps, and linear decay of the learning rate. We use a dropout probability of 0.1 on all layers. We use a gelu activation (Hendrycks and Gimpel, 2016) rather than the standard relu, following OpenAI GPT. The training loss is the sum of the mean masked LM likelihood and the mean next sentence prediction likelihood.

Training of BERT_{BASE} was performed on 4 Cloud TPUs in Pod configuration (16 TPU chips total).¹³ Training of BERT_{LARGE} was performed on 16 Cloud TPUs (64 TPU chips total). Each pre-training took 4 days to complete.

Longer sequences are disproportionately expensive because attention is quadratic to the sequence length. To speed up pretraining in our experiments, we pre-train the model with sequence length of 128 for 90% of the steps. Then, we train the rest 10% of the steps of sequence of 512 to learn the positional embeddings.

A.3 Fine-tuning Procedure

For fine-tuning, most model hyperparameters are the same as in pre-training, with the exception of the batch size, learning rate, and number of training epochs. The dropout probability was always kept at 0.1. The optimal hyperparameter values are task-specific, but we found the following range of possible values to work well across all tasks:

- **Batch size:** 16, 32

¹³<https://cloudplatform.googleblog.com/2018/06/Cloud-TPU-now-offers-preemptible-pricing-and-global-availability.html>

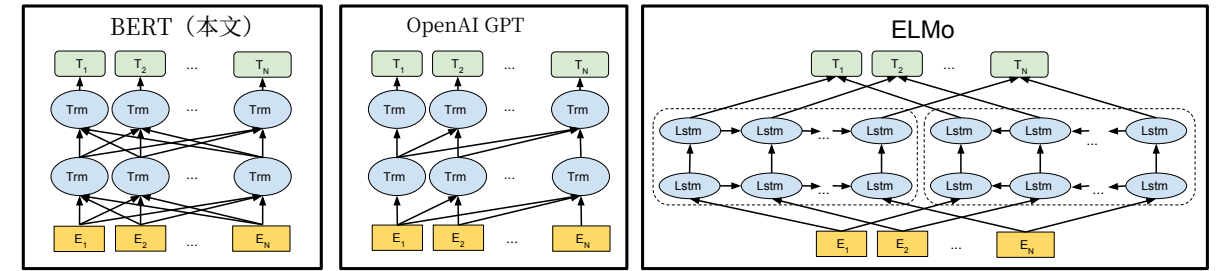


图3：预训练模型架构的差异。BERT 采用双向 Transformer。OpenAI GPT 采用从左至右的 Transformer。ELMo 使用独立训练的从左至右和从右至左 LSTMs 的拼接结果，为下游任务生成特征。在这三者中，仅 BERT 的表征在所有层中均同时以左右两侧上下文作为条件。除架构差异外，BERT 和 OpenAI GPT 属于微调方法，而 ELMo 属于基于特征的方法。

趋于收敛。在章节C.1 我们证明，MLM 确实比从左到右模型（即预测每个标记的模型）收敛得稍慢一些，但其带来的实证提升远远超过了增加的训练成本。

下一句预测 下一句预测任务可通过以下示例进行说明。

Input = [CLS] 男人去了 [MASK] 商店 [SEP]他买了一加仑 [MASK] 牛奶 [SEP]
Label = IsNext
Input = [CLS] 男人 [MASK] 去商店 [SEP]企鹅 [MASK] 是不擅长飞行的鸟类 [SEP]
Label = NotNext

A.2 预训练流程

为了生成每个训练输入序列，我们从语料库中采样两段文本，尽管它们通常比单个句子长得多（有时也可能更短），我们仍将其称为“句子”。第一个句子接收 A 嵌入，第二个句子接收 B 嵌入。50% 的情况下，B 是紧跟在 A 之后的真实下一个句子；另外 50% 的情况下，B 是随机抽取的句子，该步骤旨在完成“下一句预测”任务。我们对它们进行采样，使其组合长度恰好为 ≤ 512 个 token。在通过 WordPiece 算法完成分词后应用掩码机制，统一采用 15% 的掩码率，且不对不完整的词块作特殊处理。

我们以 256 个序列的批大小（256个序列 * 512 个词元 = 128,000 个词元/批次）进行训练，持续 1,000,000 步，这大约相当于 40

在 33 亿词的语料库上训练约 40 个 epoch。我们使用 Adam，学习率为 $1e-4$ ， $\beta_1 = 0.9$ ， $\beta_2 = 0.999$ ，L2 权重衰减系数为 0.01，在前 10,000 步中进行学习率预热，并采用线性学习率衰减。我们在所有层上使用概率为 0.1 的 Dropout。我们使用 GELU 激活函数（Hendrycks and Gimpel, 2016）而非标准 ReLU，遵循 OpenAI GPT 的做法。训练损失为平均掩码语言模型似然与平均下一句预测似然之和。

BERT_{BASE} 的训练是在 4 台采用 Pod 配置的 Cloud TPU 上进行的（总计 16 枚 TPU 芯片）。¹³ BERT_{LARGE} 的训练则在 16 台 Cloud TPU 上执行（总计 64 枚 TPU 芯片）。每次预训练均耗时 4 天完成。

更长的序列计算成本不成比例地高昂，因为注意力机制的计算复杂度与序列长度呈二次方关系。为加速实验中的预训练过程，我们在 90% 的训练步骤中采用长度为 128 的序列对模型进行预训练。随后，我们在剩余的 10% 步骤中使用长度为 512 的序列进行训练，以学习位置嵌入。

A.3 微调流程

对于微调而言，大多数模型超参数与预训练时保持一致，不同之处在于批量大小、学习率和训练轮数。随机失活（Dropout）概率始终保持在 0.1。最优超参数值具有任务特异性，但我们发现以下取值范围在所有任务中均能取得良好效果：

- **批大小:** 16, 32

¹³<https://cloudplatform.googleblog.com/2018/06/Cloud-TPU-now-offers-preemptible-pricing-and-global-availability.html>

- **Learning rate (Adam):** 5e-5, 3e-5, 2e-5
- **Number of epochs:** 2, 3, 4

We also observed that large data sets (e.g., 100k+ labeled training examples) were far less sensitive to hyperparameter choice than small data sets. Fine-tuning is typically very fast, so it is reasonable to simply run an exhaustive search over the above parameters and choose the model that performs best on the development set.

A.4 Comparison of BERT, ELMo ,and OpenAI GPT

Here we studies the differences in recent popular representation learning models including ELMo, OpenAI GPT and BERT. The comparisons between the model architectures are shown visually in Figure 3. Note that in addition to the architecture differences, BERT and OpenAI GPT are fine-tuning approaches, while ELMo is a feature-based approach.

The most comparable existing pre-training method to BERT is OpenAI GPT, which trains a left-to-right Transformer LM on a large text corpus. In fact, many of the design decisions in BERT were intentionally made to make it as close to GPT as possible so that the two methods could be minimally compared. The core argument of this work is that the bi-directionality and the two pre-training tasks presented in Section 3.1 account for the majority of the empirical improvements, but we do note that there are several other differences between how BERT and GPT were trained:

- GPT is trained on the BooksCorpus (800M words); BERT is trained on the BooksCorpus (800M words) and Wikipedia (2,500M words).
- GPT uses a sentence separator ([SEP]) and classifier token ([CLS]) which are only introduced at fine-tuning time; BERT learns [SEP], [CLS] and sentence A/B embeddings during pre-training.
- GPT was trained for 1M steps with a batch size of 32,000 words; BERT was trained for 1M steps with a batch size of 128,000 words.
- GPT used the same learning rate of 5e-5 for all fine-tuning experiments; BERT chooses a task-specific fine-tuning learning rate which performs the best on the development set.

To isolate the effect of these differences, we perform ablation experiments in Section 5.1 which demonstrate that the majority of the improvements are in fact coming from the two pre-training tasks and the bidirectionality they enable.

A.5 Illustrations of Fine-tuning on Different Tasks

The illustration of fine-tuning BERT on different tasks can be seen in Figure 4. Our task-specific models are formed by incorporating BERT with one additional output layer, so a minimal number of parameters need to be learned from scratch. Among the tasks, (a) and (b) are sequence-level tasks while (c) and (d) are token-level tasks. In the figure, E represents the input embedding, T_i represents the contextual representation of token i , [CLS] is the special symbol for classification output, and [SEP] is the special symbol to separate non-consecutive token sequences.

B Detailed Experimental Setup

B.1 Detailed Descriptions for the GLUE Benchmark Experiments.

Our GLUE results in Table1 are obtained from <https://gluebenchmark.com/leaderboard> and <https://blog.openai.com/language-unsupervised>. The GLUE benchmark includes the following datasets, the descriptions of which were originally summarized in Wang et al. (2018a):

MNLI Multi-Genre Natural Language Inference is a large-scale, crowdsourced entailment classification task (Williams et al., 2018). Given a pair of sentences, the goal is to predict whether the second sentence is an *entailment*, *contradiction*, or *neutral* with respect to the first one.

QQP Quora Question Pairs is a binary classification task where the goal is to determine if two questions asked on Quora are semantically equivalent (Chen et al., 2018).

QNLI Question Natural Language Inference is a version of the Stanford Question Answering Dataset (Rajpurkar et al., 2016) which has been converted to a binary classification task (Wang et al., 2018a). The positive examples are (question, sentence) pairs which do contain the correct answer, and the negative examples are (question, sentence) from the same paragraph which do not contain the answer.

- **学习率 (Adam)** : 5e-5, 3e-5, 2e-5
- **训练轮数**: 2, 3, 4

我们还观察到，大规模数据集（例如 100k+ 个已标注的训练样本）相较于小规模数据集，对超参数选择的敏感度要低得多。微调过程通常非常迅速，因此直接对上述参数进行穷举搜索，并选择开发集上表现最优的模型是合理的。

A.4 BERT、ELMo与OpenAI GPT的比较

本文研究了包括ELMo、OpenAI GPT和BERT在内的近期流行表示学习模型的差异。不同模型架构的可视化对比如图3所示。需要注意的是，除架构差异外，BERT和OpenAI GPT属于微调方法，而ELMo是一种基于特征的方法。

与BERT最可比的现有预训练方法是 OpenAI GPT，它在大型文本语料库上训练从左到右的Transformer语言模型。事实上，BERT的许多设计决策均是刻意安排，以使其尽可能接近GPT，从而便于对这两种方法进行最小化比较。本工作的核心论点是，第3.1节提出的双向性及两种预训练任务解释了绝大部分实证性能提升，但我们确实注意到，BERT与GPT的训练过程存在其他若干差异：

- GPT基于BooksCorpus（800Mwords）进行训练；BERT基于BooksCorpus（800M words）和Wikipedia（2,500Mwords）进行训练。
- GPT使用句子分隔符（[SEP]）和分类标记（[CLS]），这些仅在微调时才被引入；BERT在预训练阶段学习[SEP]，[CLS]以及句子A/B的嵌入表示。
- GPT以32,000words为批次大小训练了1M步；BERT以128,000words为批次大小训练了1M步。
- GPT在所有微调实验中均使用相同的5e-5学习率；BERT则选择了在开发集上表现最佳的特定任务微调学习率。

为了隔离这些差异的影响，我们在第5.1节进行了消融实验，结果表明绝大部分提升实际上来自这两个预训练任务及其所实现的双向性。

A.5 不同任务的微调图示

微调BERT在不同任务上的示意图见图4。各特定任务模型通过在BERT基础上增加一个额外输出层构建而成，因此只需从头学习极少量参数。在这些任务中，(a)和(b)属于序列级任务，而(c)和(d)属于标记级任务。在图中， E 代表输入嵌入表示， T_i 代表标记 i ，[CLS]是用于分类输出的特殊符号，而[SEP]是用于分隔不连续标记序列的特殊符号。

B 详细的实验设置

B.1 GLUE基准实验的详细描述。

我们的GLUE结果见表1 均获取自 <https://gluebenchmark.com/leaderboard>以及 <https://blog.openai.com/language-unsupervised>。GLUE基准测试包含以下data集，其原始描述首次由Wang等人 (2018a)：

MNLI 多体裁自然语言推理是一个大规模的众包蕴含分类任务 (Williams 等人, 2018)。给定一对句子，目标是预测第二个句子相对于第一个句子是 蕴含, 矛盾, 还是中立相对于第一个句子的关系。

QQP (Quora问题对) 是一个二元分类任务，目标是判断在Quora上提出的两个问题是否在语义上等价 (Chen等, 2018)。

QNLI “问题自然语言推理” (Question Natural Language Inference) 是斯坦福问答数据集的Rajpurkar 等人, 2016版，该数据集已被转换为二分类任务 (2018a)。正例为包含正确答案的(问题, 句子)对，而负例则是同一段落中不包含答案的(问题, 句子)对。

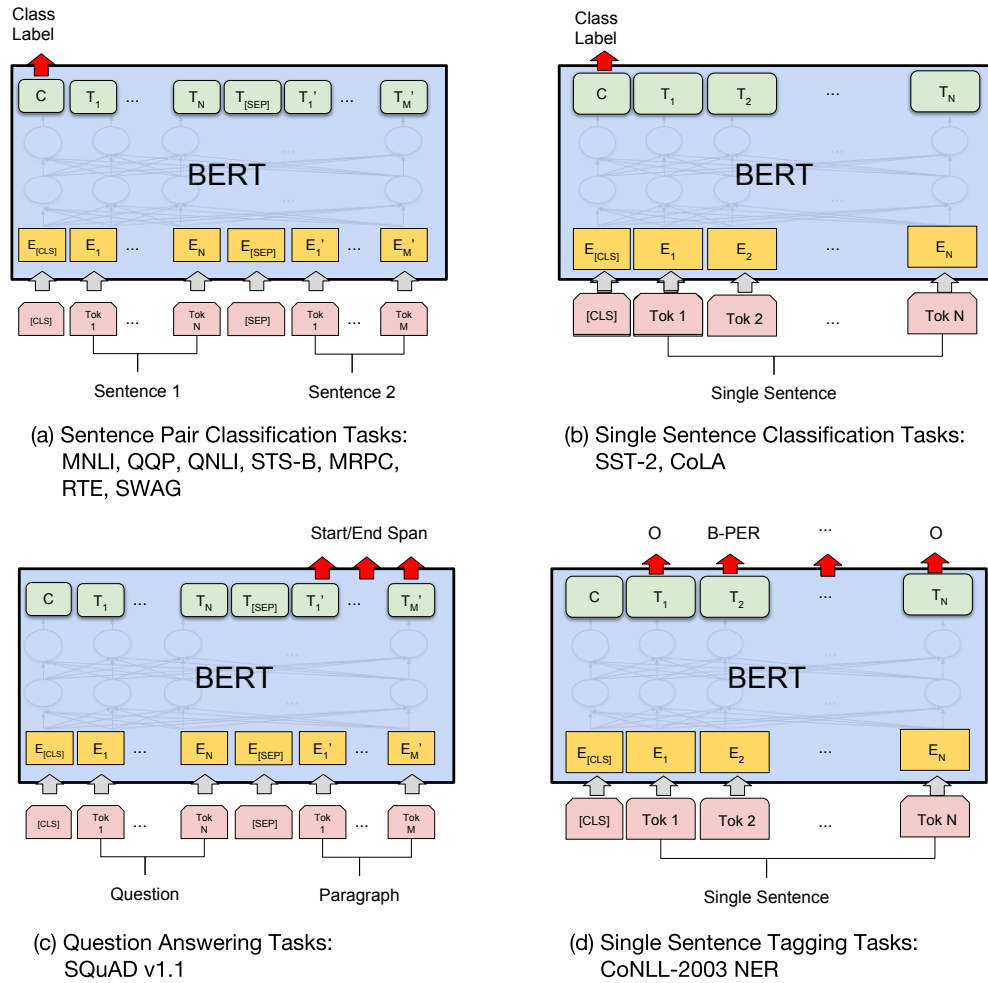


Figure 4: Illustrations of Fine-tuning BERT on Different Tasks.

SST-2 The Stanford Sentiment Treebank is a binary single-sentence classification task consisting of sentences extracted from movie reviews with human annotations of their sentiment (Socher et al., 2013).

CoLA The Corpus of Linguistic Acceptability is a binary single-sentence classification task, where the goal is to predict whether an English sentence is linguistically “acceptable” or not (Warstadt et al., 2018).

STS-B The Semantic Textual Similarity Benchmark is a collection of sentence pairs drawn from news headlines and other sources (Cer et al., 2017). They were annotated with a score from 1 to 5 denoting how similar the two sentences are in terms of semantic meaning.

MRPC Microsoft Research Paraphrase Corpus consists of sentence pairs automatically extracted from online news sources, with human annotations

for whether the sentences in the pair are semantically equivalent (Dolan and Brockett, 2005).

RTE Recognizing Textual Entailment is a binary entailment task similar to MNLI, but with much less training data (Bentivogli et al., 2009).¹⁴

WNLI Winograd NLI is a small natural language inference dataset (Levesque et al., 2011). The GLUE webpage notes that there are issues with the construction of this dataset,¹⁵ and every trained system that’s been submitted to GLUE has performed worse than the 65.1 baseline accuracy of predicting the majority class. We therefore exclude this set to be fair to OpenAI GPT. For our GLUE submission, we always predicted the ma-

¹⁴Note that we only report single-task fine-tuning results in this paper. A multitask fine-tuning approach could potentially push the performance even further. For example, we did observe substantial improvements on RTE from multitask training with MNLI.

¹⁵<https://gluebenchmark.com/faq>

SST-2 斯坦福情感树库是一个二分类的单词分类任务，由从电影评论中提取的句子组成，并附有人工情感标注 (Socher 等人,2013)。

CoLA 语言可接受性语料库是一个二分类单词分类任务，其目标是判断英文句子在语言学上是否“可接受” (Warstadt et al.,2018)。

STS-B 语义文本相似度基准是一组句子对的集合，这些句子对选自新闻标题及其他来源 (Cer 等,2017)。它们的评分为 1 至 5 分，用于表示两个句子在语义意义上的相似程度。

MRPC 微软研究释义语料库由自动从在线新闻来源提取的句子对组成，并附有人工标注

用于判断该句对是否语义等价 (Dolan与 Brockett, 2005)。

RTE 文本蕴含识别是一项二元蕴含任务，与 MNLI 类似，但训练数据量大幅减少 (Bentivogli 等, 2009)。¹⁴

WNLI Winograd NLI 是一个小型自然语言推理数据集 (Levesque 等, 2011年)。GLUE 网页指出该数据集的构建存在一些问题，¹⁵ 且所有提交至 GLUE 的训练系统表现均低于预测多数类所取得的 65.1 基线准确率。因此，为求公平起见，我们排除此数据集。对于我们的 GLUE 提交，我们始终预测主-

¹⁴请注意，本文仅报告单任务微调的结果。多任务微调方法有望进一步推升性能。例如，我们确实观察到通过结合 MNLI 进行多任务训练，RTE 取得了显著提升。

¹⁵<https://gluebenchmark.com/faq>

图4：在不同任务上微调BERT的示意图。

jority class.

C Additional Ablation Studies

C.1 Effect of Number of Training Steps

Figure 5 presents MNLI Dev accuracy after fine-tuning from a checkpoint that has been pre-trained for k steps. This allows us to answer the following questions:

1. Question: Does BERT really need such a large amount of pre-training (128,000 words/batch * 1,000,000 steps) to achieve high fine-tuning accuracy?
Answer: Yes, BERT_{BASE} achieves almost 1.0% additional accuracy on MNLI when trained on 1M steps compared to 500k steps.
2. Question: Does MLM pre-training converge slower than LTR pre-training, since only 15% of words are predicted in each batch rather than every word?

Answer: The MLM model does converge slightly slower than the LTR model. However, in terms of absolute accuracy the MLM model begins to outperform the LTR model almost immediately.

C.2 Ablation for Different Masking Procedures

In Section 3.1, we mention that BERT uses a mixed strategy for masking the target tokens when pre-training with the masked language model (MLM) objective. The following is an ablation study to evaluate the effect of different masking strategies.

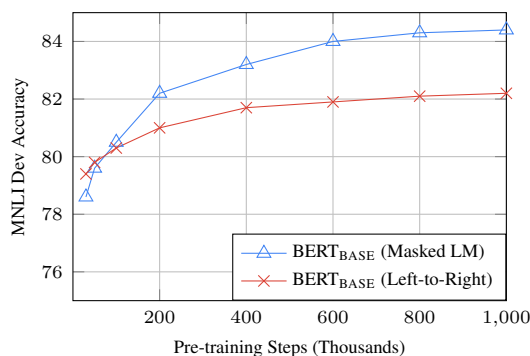


Figure 5: Ablation over number of training steps. This shows the MNLI accuracy after fine-tuning, starting from model parameters that have been pre-trained for k steps. The x-axis is the value of k .

Note that the purpose of the masking strategies is to reduce the mismatch between pre-training and fine-tuning, as the [MASK] symbol never appears during the fine-tuning stage. We report the Dev results for both MNLI and NER. For NER, we report both fine-tuning and feature-based approaches, as we expect the mismatch will be amplified for the feature-based approach as the model will not have the chance to adjust the representations.

Masking Rates			Dev Set Results		
MASK	SAME	RND	MNLI Fine-tune	NER Fine-tune	NER Feature-based
80%	10%	10%	84.2	95.4	94.9
100%	0%	0%	84.3	94.9	94.0
80%	0%	20%	84.1	95.2	94.6
80%	20%	0%	84.4	95.2	94.7
0%	20%	80%	83.7	94.8	94.6
0%	0%	100%	83.6	94.9	94.6

Table 8: Ablation over different masking strategies.

The results are presented in Table 8. In the table, MASK means that we replace the target token with the [MASK] symbol for MLM; SAME means that we keep the target token as is; RND means that we replace the target token with another random token.

The numbers in the left part of the table represent the probabilities of the specific strategies used during MLM pre-training (BERT uses 80%, 10%, 10%). The right part of the paper represents the Dev set results. For the feature-based approach, we concatenate the last 4 layers of BERT as the features, which was shown to be the best approach in Section 5.3.

From the table it can be seen that fine-tuning is surprisingly robust to different masking strategies. However, as expected, using only the MASK strategy was problematic when applying the feature-based approach to NER. Interestingly, using only the RND strategy performs much worse than our strategy as well.

多数类。

C 补充消融实验

C.1 训练步数的影响

图5展示了通过微调预训练检查点获得的 MNLI Dev 准确率，该模型已预训练 k 步。这使我们能够回答以下问题：

1. 问题：BERT 是否真的需要如此大规模的预训练（每批 128,000词 × 1,000,000 步）才能达到高微调精度？
回答：是的，相较于 500k 步，在 1M 步上进行训练时，BERT_{BASE} 在 MNLI 上的准确率几乎获得了额外的 1.0% 提升。
2. 问题：由于每个批次仅预测 15% 的词而非全部，MLM 预训练的收敛速度是否比 LTR 预训练更慢？

答案：MLM 模型的收敛速度确实比 LTR 模型稍慢。然而，在绝对精度方面，MLM 模型几乎从一开始就超越 LTR 模型。

C.2 不同掩码方法的消融实验

在第3.1节中，我们指出 BERT 在使用掩码语言模型（MLM）目标进行预训练时，采用了混合策略来掩盖目标词元。以下是一项消融研究，旨在评估不同掩盖策略的效果。

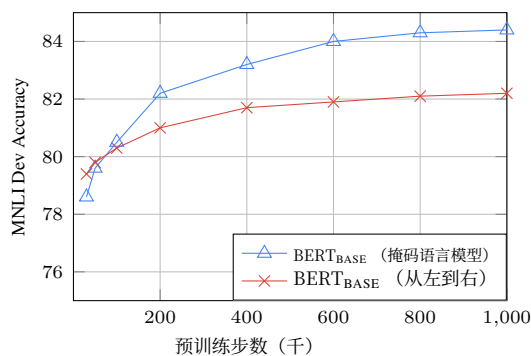


图5：不同训练步数的消融实验。该图展示了微调后的 MNLI 准确率，起点为已预训练 k 步的模型参数。横轴表示 k 的值。

需要注意的是，这些掩盖策略的目的是为了降低预训练与微调之间的不匹配程度，因为 [MASK] 标记在微调阶段根本不会出现。我们汇报了 MNLI 和 NER 任务在开发集（Dev）上的结果。针对 NER 任务，我们同时报告了微调 and 基于特征的方法的结果；我们预计基于特征的方法受这种不匹配的影响会更显著，因为该模型的表示无法在此阶段进行调整。

掩码率			验证集结果		
掩码相同	RND		MNLI 微调	NER 微调	NER 基于特征的
80%	10%	10%	84.2	95.4	94.9
100%	0%	0%	84.3	94.9	94.0
80%	0%	20%	84.1	95.2	94.6
80%	20%	0%	84.4	95.2	94.7
0%	20%	80%	83.7	94.8	94.6
0%	0%	100%	83.6	94.9	94.6

表8：不同掩码策略的消融实验。

结果如表8所示。表中，MASK表示在 MLM中将目标词元替换为 [MASK] 符号；SAME表示保持目标词元原样；RND表示将目标词元替换为另一个随机词元。

表左侧的数值表示在MLM预训练过程中采用各特定策略的概率（BERT采用的比例为80%、10%和10%）。表格右侧部分展示的是 Dev集的测试结果。针对基于特征的方法，我们将BERT的最后4层进行拼接以获取特征，该方法在第5.3节已被证明为最优方案。

从表中可以看出，微调对不同掩码策略展现出令人惊讶的鲁棒性。然而，正如预期，在将基于特征的方法应用于命名实体识别（NER）任务时，仅使用MASK策略会带来问题。有趣的是，仅使用RND策略的表现也远逊于我们的策略。